



1장 머신러닝과 딥러닝 & 2장 실습 환경 설정과 파이토치 기초

1팀 | 이은수 안서영 조현아

목차

01 머신 러닝과 딥러닝

인공지능, 머신 러닝과 딥러닝
머신 러닝이란
딥러닝이란

02 실습 환경 설정과 파이토치 기초

파이토치 개요
파이토치 기초 문법
실습 환경 설정
파이토치 코드 맛보기



1. 머신 러닝과 딥러닝



#01 인공지능, 머신 러닝과 딥러닝

인공지능이란?

: 인간의 지능을 모방하여, 사람이 하는 일을 컴퓨터(기계)가 할 수 있도록 하는 기술

구현 방법

01 머신러닝

사람이 데이터의 특징을 직접 뽑아주고, 적절한 알고리즘을 선택해 학습시키는 방법

02 딥러닝

신경망이 데이터의 특징을 스스로 자동으로 학습하는 방법

인공지능 & 머신러닝 & 딥러닝의 관계

인공지능(AI)



머신러닝(ML)



딥러닝(DL)

#01 인공지능, 머신 러닝과 딥러닝

머신러닝 vs 딥러닝

공통점

: 학습 모델 -> 데이터를 분류

차이점

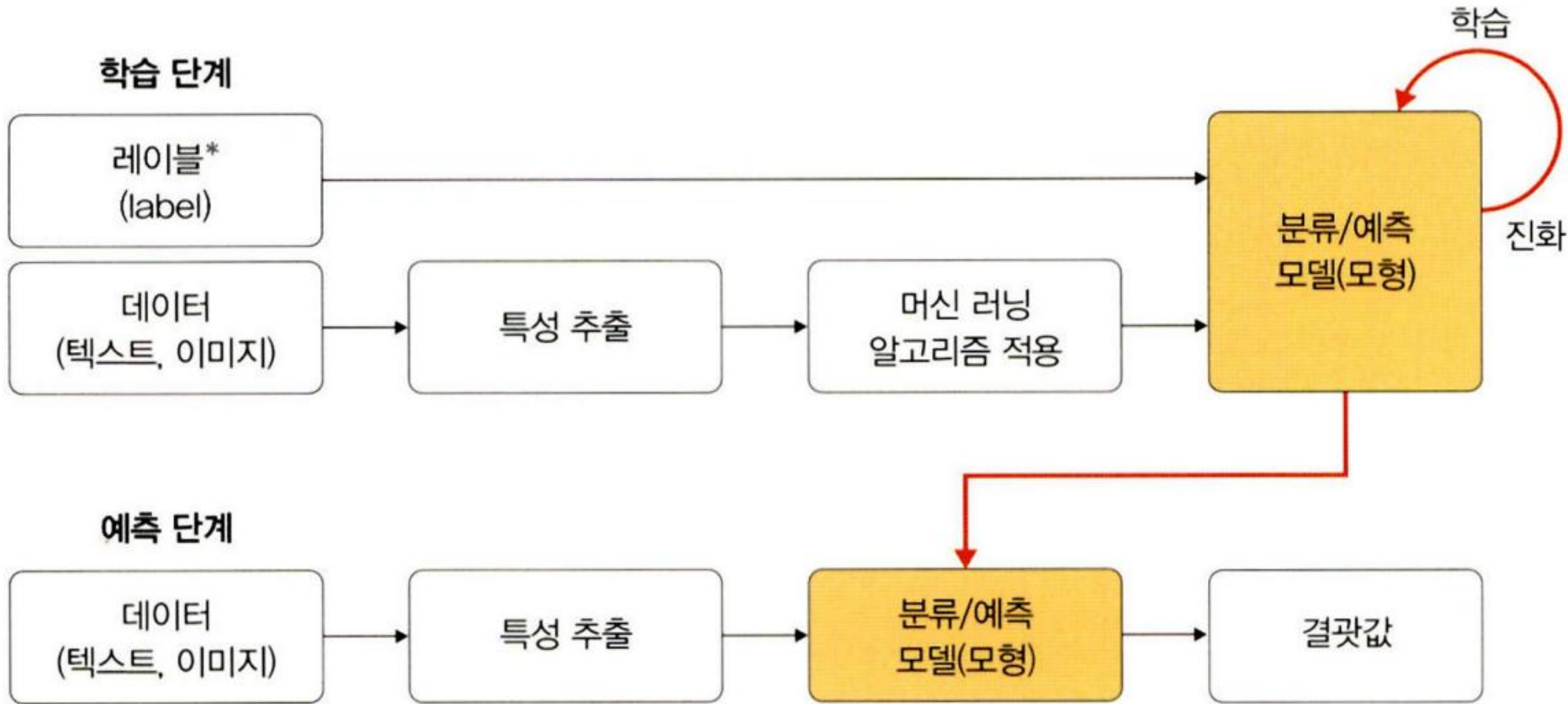
구분	머신러닝	딥러닝
동작 원리	입력 데이터에 머신러닝 알고리즘을 적용하여 예측	신경망을 이용하여 데이터의 특징과 관계를 학습하고 예측
특징 추출 / 전처리	사람이 데이터의 특징을 직접 선택·전처리한 후 학습	신경망이 데이터에서 특징을 자동으로 추출하며 학습
데이터	비교적 적은 데이터로도 학습 가능	일반적으로 많은 데이터가 필요
알고리즘	문제에 따라 Linear Regression, Decision Tree, SVM 등 다양한 알고리즘을 선택	인공신경망을 기반으로 여러 층의 신경망을 구성
훈련시간	비교적 짧음	비교적 김
재사용	학습된 모델을 새로운 데이터에 적용할 수 있으나, 다른 문제에 활용하려면 별도의 학습이 필요한 경우가 많음	사전 학습 모델(Pre-trained Model)을 새로운 유사 문제에 활용하는 것이 학습이 가능
결과	분류, 회귀 등의 예측 결과를 출력	이미지, 텍스트, 음성 등 다양한 형태의 데이터를 처리하고 결과를 출력

#02 머신 러닝

머신러닝이란

컴퓨터 스스로 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측을 수행 — 이를 가능하게 하는 알고리즘과 기술을 개발하는 분야

학습 과정



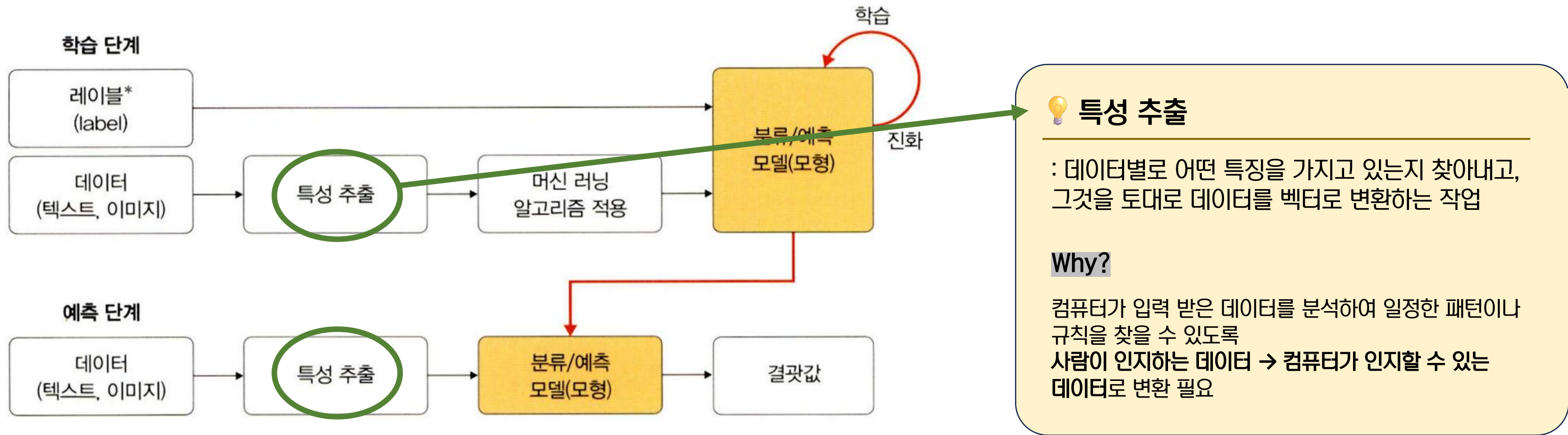
* 레이블은 지도 학습에서 정답을 의미

#02 머신 러닝

머신러닝이란

컴퓨터 스스로 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측을 수행 — 이를 가능하게 하는 알고리즘과 기술을 개발하는 분야

학습 과정



* 레이블은 지도 학습에서 정답을 의미

#02 머신 러닝

머신러닝의 구성 요소

01 데이터

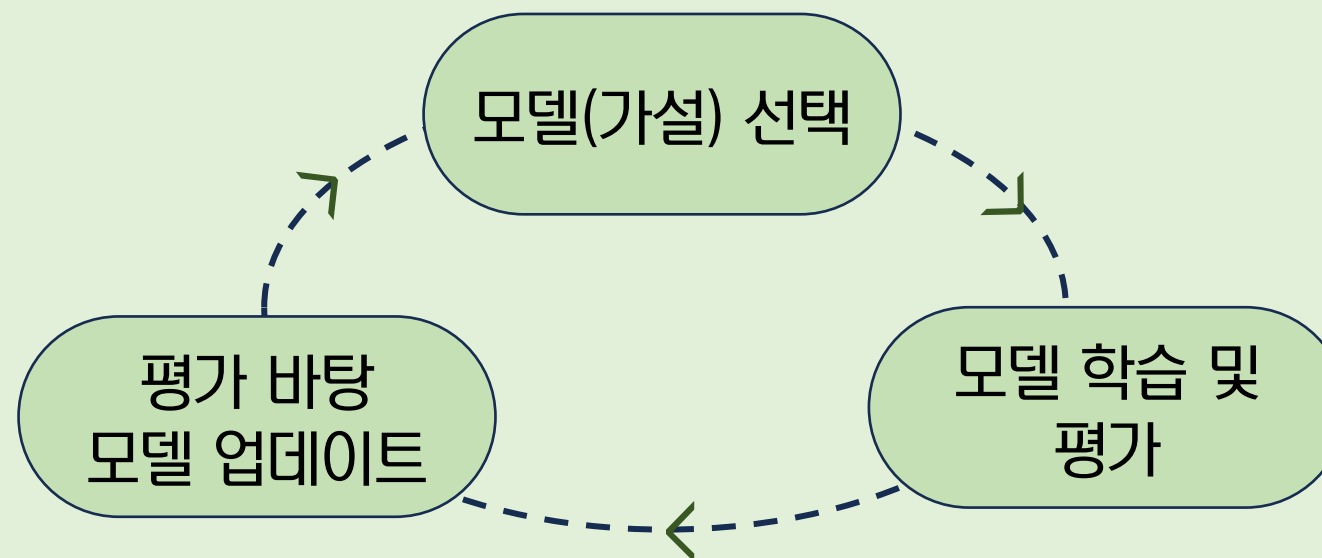
- 학습 모델을 만드는 데 사용되는 재료
- 실제 데이터의 특징이 잘 반영되고 편향X인 학습 데이터 확보가 중요
- 훈련 / 테스트 / 검증 데이터 셋으로 분리

💡 검증 데이터 셋이 필요한 이유

1. 테스트 데이터셋에 대한 성능 가늠 가능
2. 모델 성능 향상에 도움이 됨
Ex - 과적합 발견 가능

02 모델

머신 러닝의 학습 단계에서 얻은 최종 결과물(=가설)

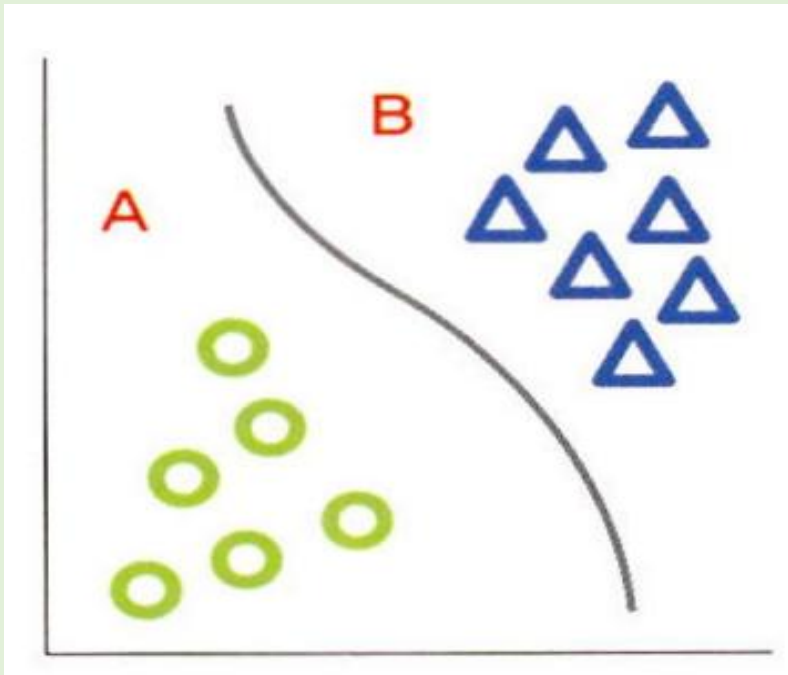


#02 머신 러닝

머신러닝 학습 알고리즘

지도학습

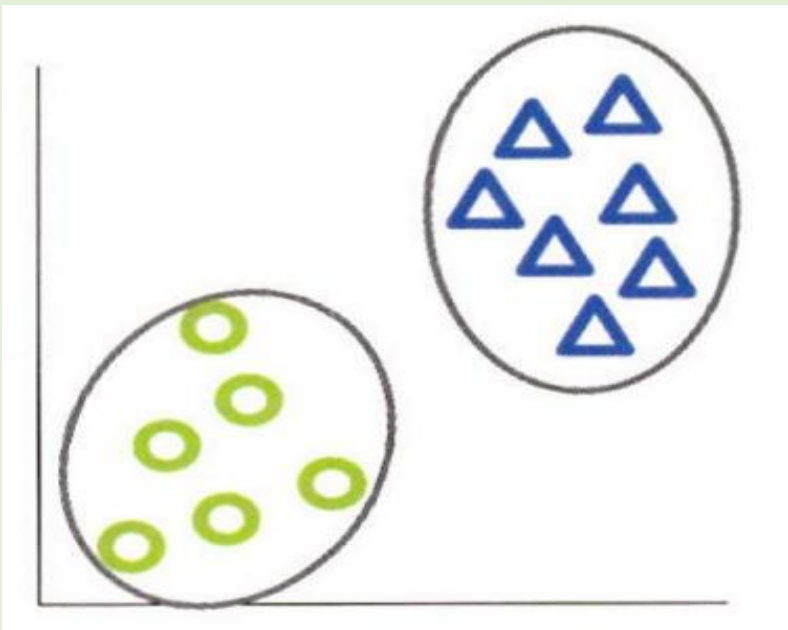
레이블(정답)이 있는 데이터로 컴퓨터를 학습시킴



비지도학습

레이블이 없는 데이터로 컴퓨터를 학습시킴

특징이 비슷한 데이터를 클러스터링(범주화)하여 예측



강화학습

정답이 없는 대신, 행동에 대한 보상을 받으며 학습 진행



보상이 커지는 행동을 자주 하도록 하고, 줄어드는 행동은 덜 하도록

#02 머신 러닝

머신러닝 대표 알고리즘

구분	유형	알고리즘
지도학습	분류	• KNN • 서포트 벡터 머신(SVM) • 결정 트리 • 로지스틱 회귀
	회귀	선형 회귀
비지도학습	군집	• K-평균 군집화 • 밀도 기반 군집 분석(DBSCAN)
	차원축소	주성분 분석(PCA)
강화학습	-	마르코프 결정 과정 (Markov Decision Process, MDP)

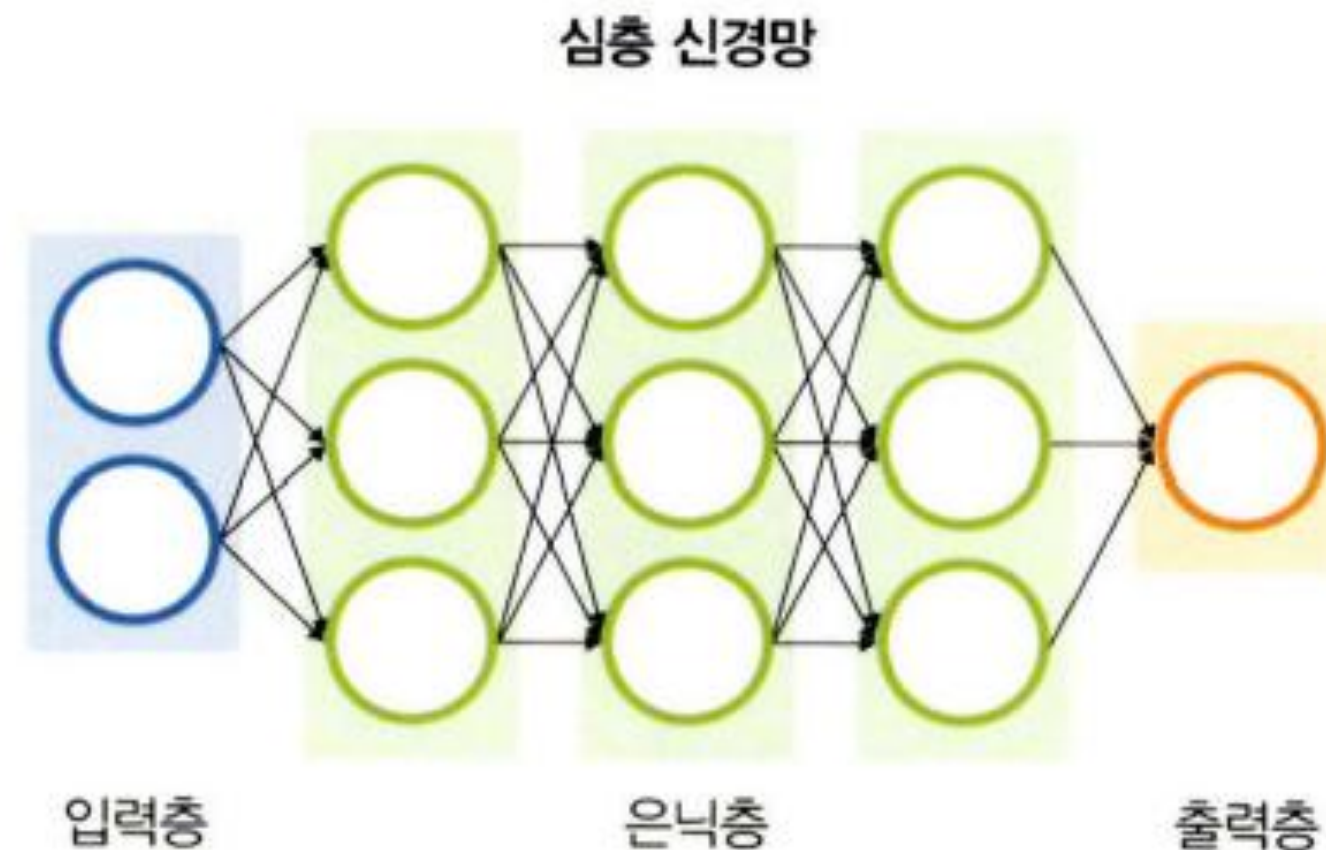
#03 딥러닝

딥러닝이란

인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신러닝 방법의 한 종류
머신러닝과 달리 인간의 뇌 구조를 기초로 설계됨

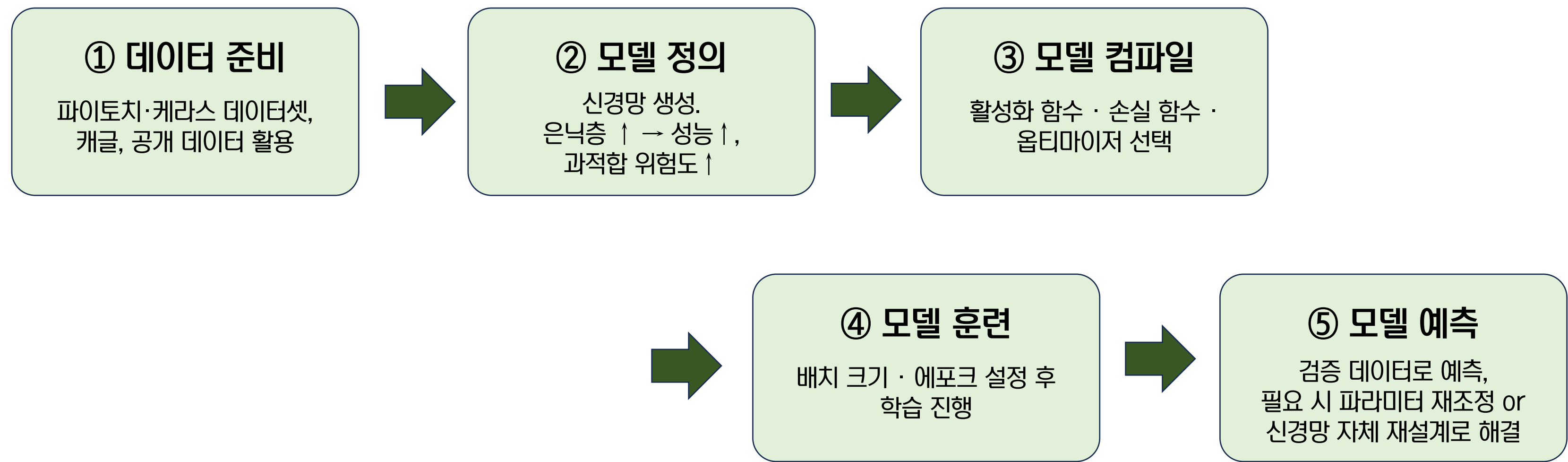
특징

- 컴퓨터에 뉴런과 시냅스 개념을 적용
- 노드(뉴런)들이 엣지(시냅스)로 연결된 신경망 구조
- 수많은 뉴런의 연산이 병렬로 이루어져 음성·영상 인식 같은 복잡한 처리가 가능



#03 딥러닝

딥러닝 학습 과정



#03 딥러닝

배치 · 에포크 · 이터레이션

배치 크기(Batch Size)

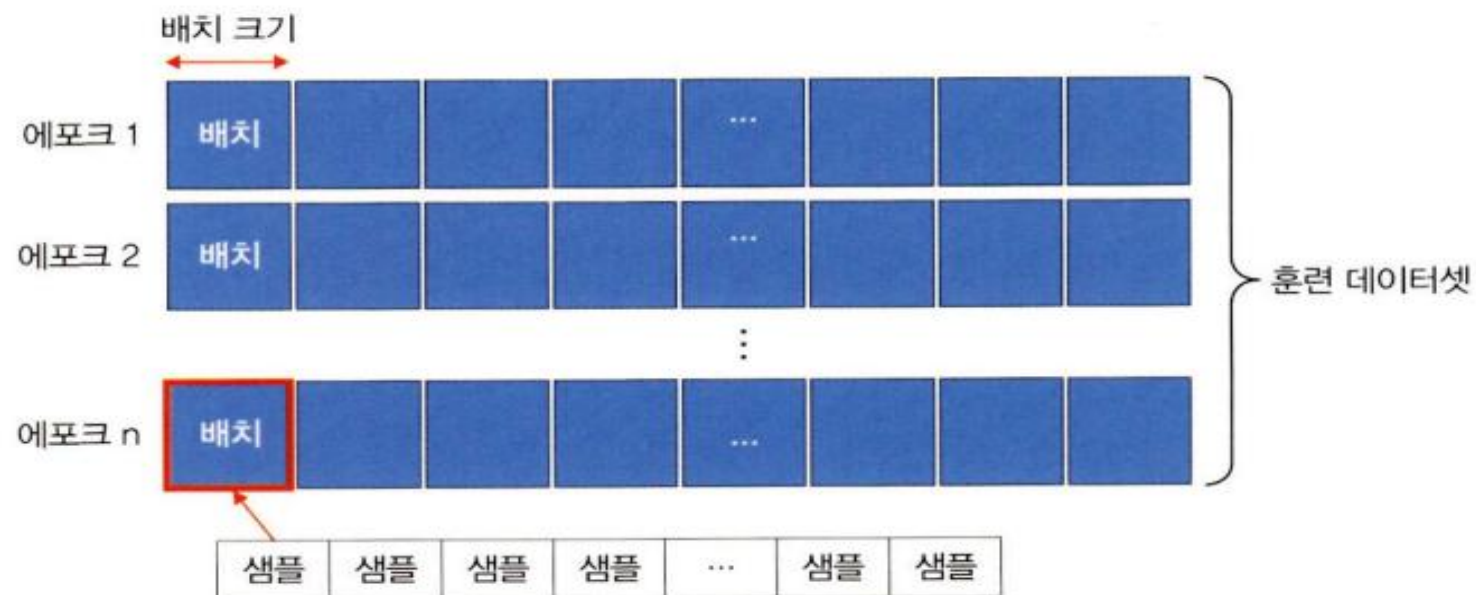
한 번에 처리할 데이터 개수.
너무 크면 메모리 부족, 너무 작으면 학습이 비효율적일 수 있음

에포크 (Epoch)

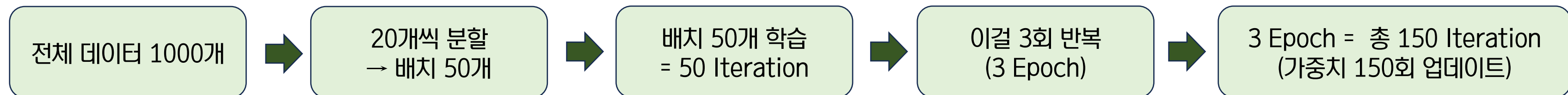
전체 훈련 데이터를 몇 번 반복해서 학습할지.
10 Epoch = 전체 데이터를 10번 학습

이터레이션 (Iteration)

하나의 배치를 학습하고 가중치를 한 번 업데이트하는 과정



예시: 전체 데이터 1,000개 · 배치 크기 20 · 3 Epoch

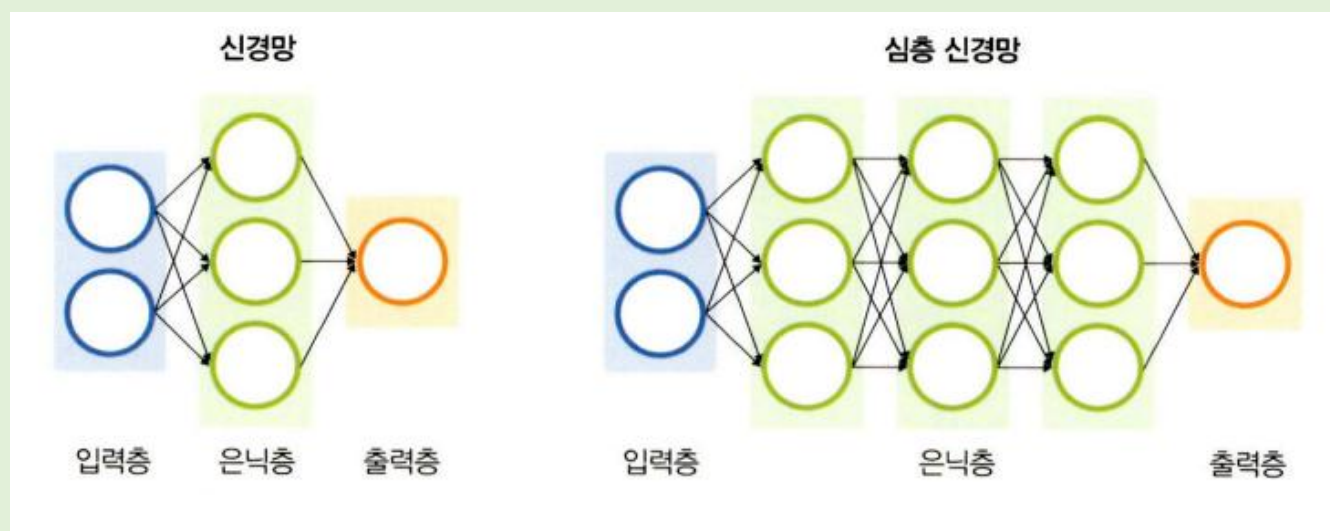


#03 딥러닝

딥러닝의 핵심 구성 요소

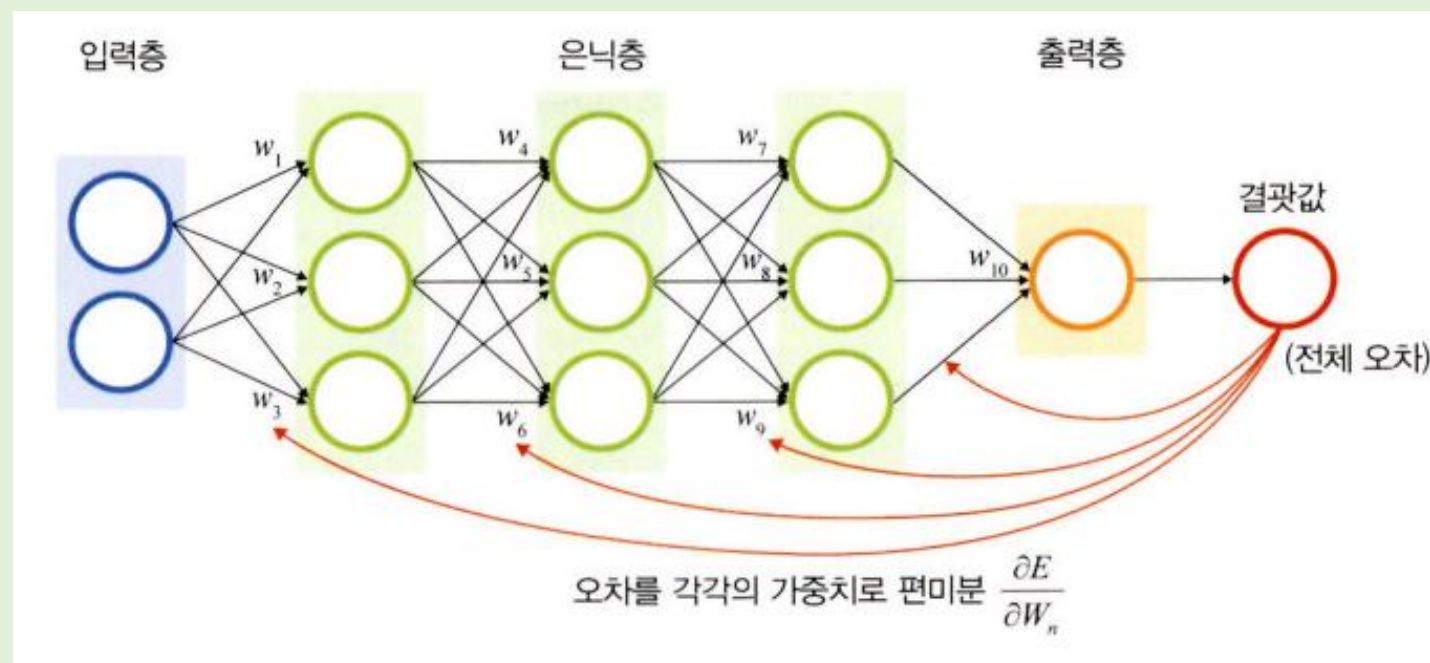
01 심층 신경망

- 머신러닝과의 차이점
- 데이터셋의 어떤 특성이 중요한지
- 스스로 학습할 수 있는 기능
- 층을 깊게 쌓아 복잡한 패턴을 표현



02 역전파

- 가중치 값을 업데이트하는 과정
- 미분(그래디언트)를 사용하여 계산
- 파이토치에서는 자동처리 → 구현이 간단해짐



#03 딥러닝

딥러닝 지도 학습

이미지 분류

- 이미지 또는 비디오상의 객체를 식별하는 컴퓨터 비전 기술
- 합성곱 신경망(CNN) 많이 사용

목적에 따라

- 이미지 분류: 입력 이미지가 어떤 클래스에 속하는지 판별
- 이미지 인식: 사물의 종류를 인식 – 의료영상 질병 식별, 로봇 비전 등에 활용
- 이미지 분할: 픽셀 단위로 객체·배경을 구분
 - X-ray ·CT·MRI 진단에 활용

시계열 데이터 분석

RNN(순환 신경망)

: 역전파 과정에서 기울기 소멸 문제 발생



LSTM

: 3개의 망각/입력/출력 게이트를 추가
→ 기울기 소멸 문제 해결
현재 시계열 문제에서 가장 활발히 사용

#03 딥러닝

딥러닝 비지도 학습

워드 임베딩

- 단어를 벡터로 표현하는 방법. 자연어를 컴퓨터가 이해하고 효율적으로 처리할 수 있도록 변환
- Word2Vec, GloVe

군집

- 아무 정보 없이 데이터를 분류
- 한 클러스터 안의 데이터는 비슷하게, 다른 클러스터와는 구분되도록 나눔
- 딥러닝을 이용해 복잡한 데이터에서 유용한 특징을 추출하여 군집화 성능 향상 가능

딥러닝 전이 학습

- 이미 많은 데이터로 학습된 사전 학습 모델(Pre-trained Model)을 새로운 문제에 활용하는 방법
- 처음부터 학습하는 것보다 데이터·학습 시간·연산량을 줄일 수 있음

활용 방법

- 특성 추출
- 미세 조정

대표적인 사전 학습 모델

: VGG, Inception, MobileNet

#03 딥러닝

딥러닝 대표 알고리즘

구분	유형	알고리즘
지도 학습	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습	군집	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습		마르코프 결정 과정(MDP)

2.1 파이토치 개요



2.1.1 파이토치 특징 및 장점

파이토치: GPU에서 텐서 조작과 동적 신경망 구축이 가능한 딥러닝 프레임워크

텐서 (Tensor)

- 파이토치가 다루는 기본 데이터 형태
- 단일 자료형으로 이루어진 다차원 행렬
- 넘파이 ndarray와 사용법이 거의 같음
- 간단한 명령어만으로 GPU 연산 가능

```
import torch  
torch.tensor([[1., -1.], [1., -1.]])
```

```
tensor([[ 1., -1.],  
        [1., -1.]])
```

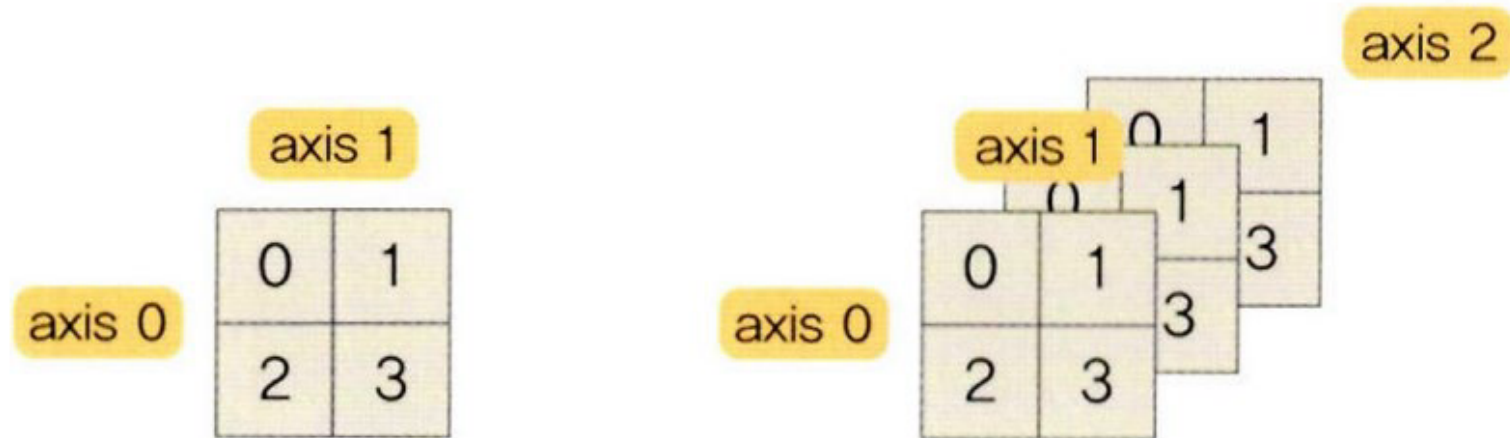


그림 2-2 벡터, 행렬, 텐서

2.1.1 파이토치 특징 및 장점

GPU 연산

병렬 연산에서 CPU보다 훨씬 빠름

딥러닝은 기울기 계산(미분)의 반복; GPU는 미분 연산 속도를 크게 끌어올림

동적 신경망

훈련을 반복하면서 네트워크를 바꿀 수 있는 신경망

학습 도중 은닉층 추가 / 제거 가능

그래프 연산과 값 초기화를 동시에 하는 Define by Run 방식

2.1.1 Define by Run

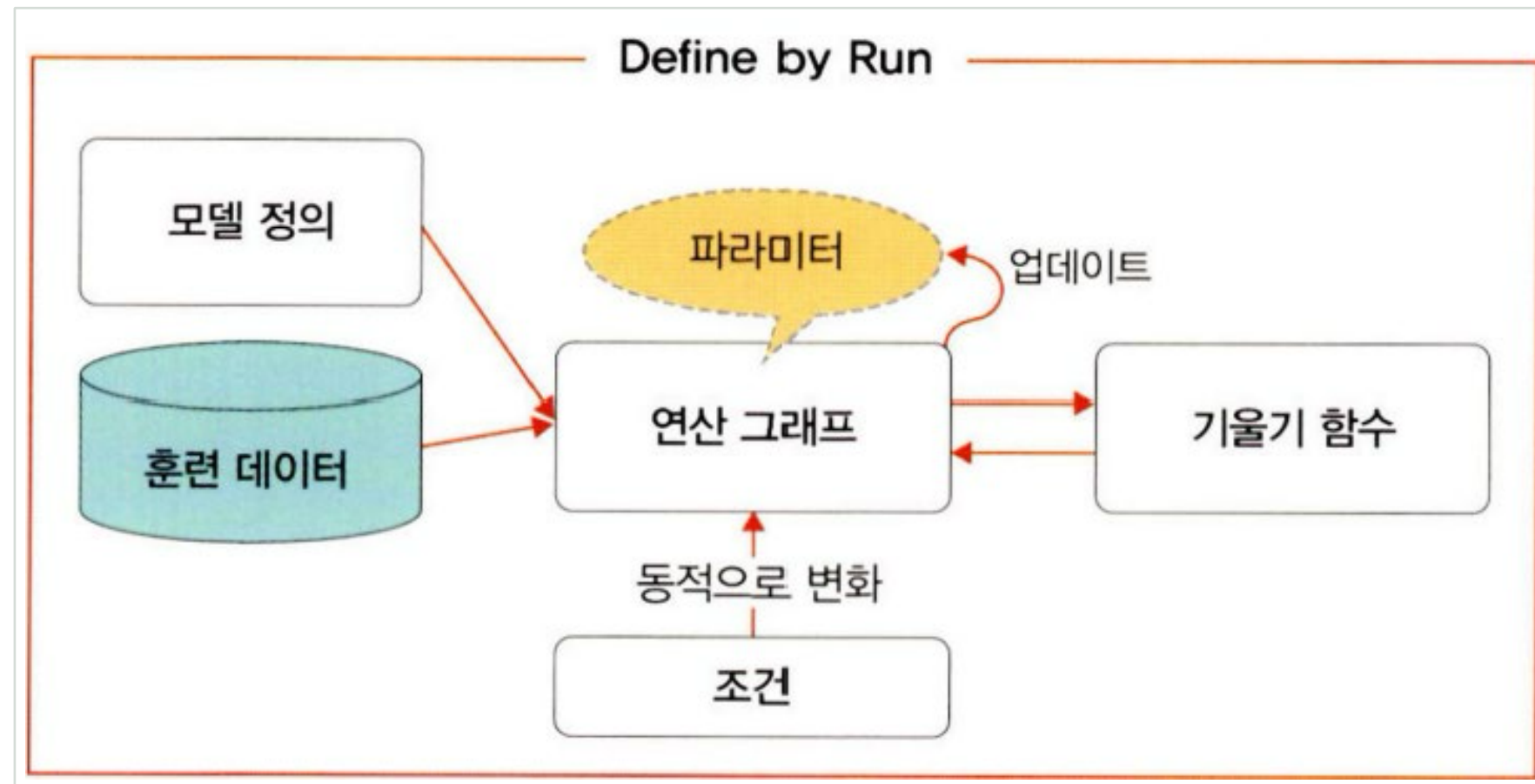


그림 2-1 Define by Run 조건에 따라 연산 그래프가 실행 중에 바뀐다

Define by Run

- 그래프를 미리 정의하지 않고, 코드가 실행되는 순간 그래프가 만들어짐
- 조건문/ 반복문에 따라 그래프가 동적으로 변함
- 학습 도중 은닉층 추가 · 제거도 가능
- 파이썬 코드 그대로 디버깅할 수 있어 직관적

2.1.1 데이터의 표현



스칼라 (0D)

shape ()

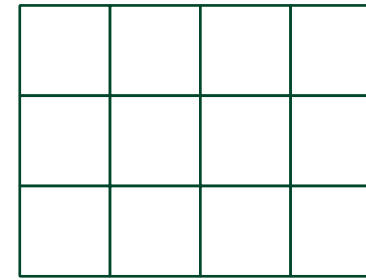
값 하나



벡터 (1D)

shape (4,)

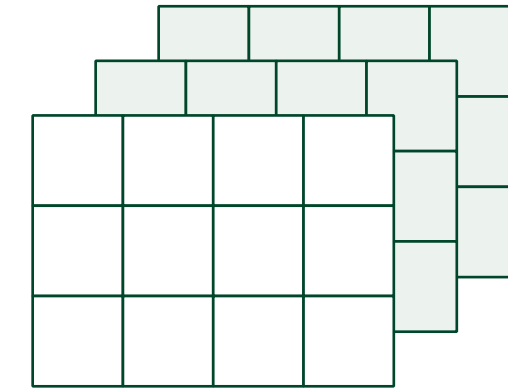
axis 0 (행)



행렬 (2D)

shape (3, 4)

axis 0 · axis 1 (행 × 열)



텐서 (3D 이상)

shape (2, 3, 4)

axis 2 (채널) 추가

- 인공지능에서 데이터는 기본적으로 벡터(vector)로 표현된다
 - 1차원 축(행) = axis 0 → 벡터 / 2차원 축(열) = axis 1 → 행렬 / 3차원 축(채널) = axis 2 → 텐서
- 행렬은 복수의 차원을 가지는 데이터 레코드의 집합. 하나의 데이터 레코드가 하나의 행이 된다
 - 텐서는 행렬의 다차원 표현
 - 파이토치는 모든 데이터를 텐서로 다룬다

2.1.1 GPU 활용

- 딥러닝 학습 = 기울기(gradient) 계산의 반복
 - 가중치마다 미분을 수행; 단순 연산이 매우 많음
- GPU는 단순 연산을 병렬로 처리하는 데 특화
 - CPU보다 훨씬 빠르게 처리

```
import torch

# CPU에 2차원 텐서 생성
print(torch.tensor([[1, 2], [3, 4]]))

# GPU에 텐서 생성
print(torch.tensor([[1, 2], [3, 4]],
                    device="cuda:0"))
```

2.1.1 연산 그래프

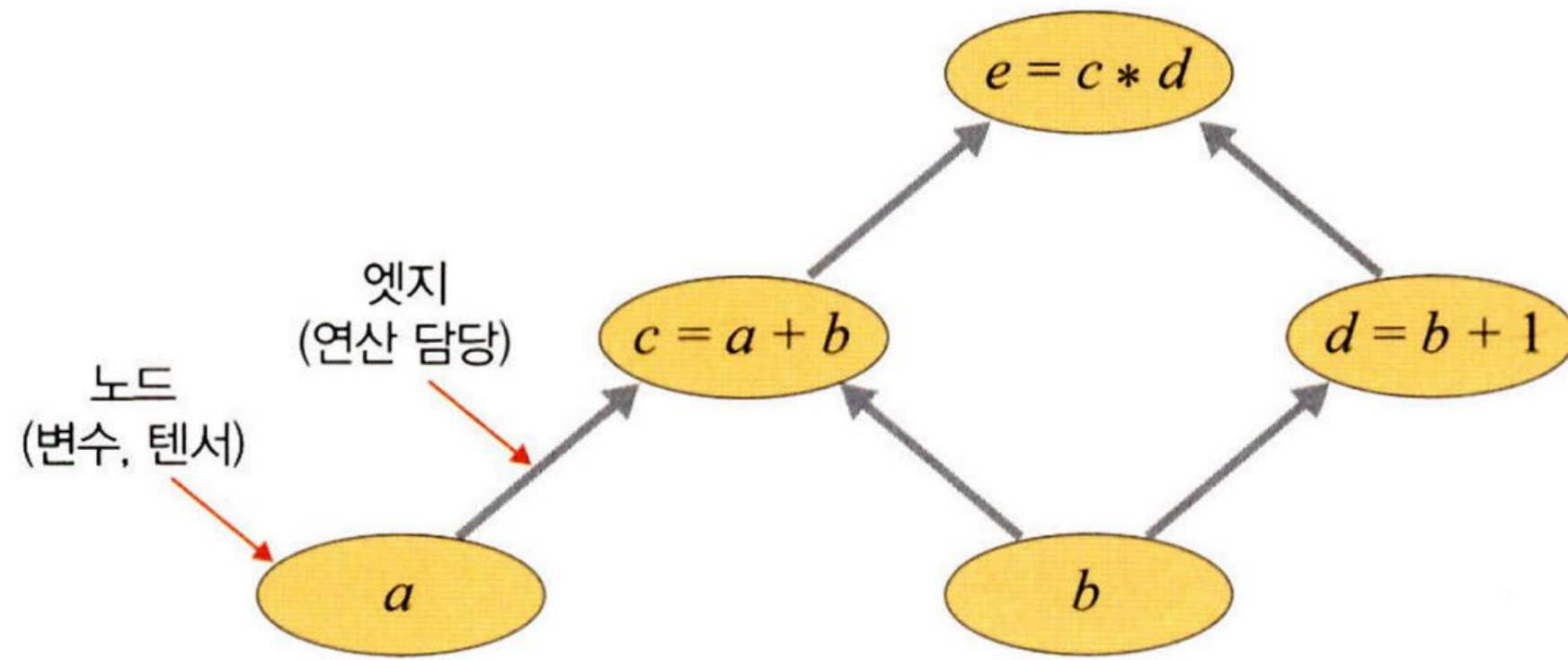


그림 2-3 파이토치 연산 그래프

연산 그래프

- 방향성이 있는 그래프
 - 노드(node) = 변수, 엣지(edge) = 연산
- 손실 함수의 기울기를 가중치 / 바이어스 기준으로 계산
 - 경사 하강법으로 파라미터를 업데이트
- 연산 그래프를 이용해 학습 과정을 효율적으로 수행

2.1.1 특징 및 장점

1. 단순함

- 파이썬 환경과 통합 용이, 넘파이를 쓰듯이 텐서를 다룰 수 있음
- 실행 중 값을 그대로 확인할 수 있어 디버깅이 직관적

2. 성능

- 모델 훈련 시 CPU 사용률이 텐서플로에 비해 낮음
- 학습 / 추론 속도가 빠름

3. 직관적 인터페이스

- API 구조가 명확하고 단순
- nn.Module 상속만으로 모델 정의

2.1.2 아키텍처

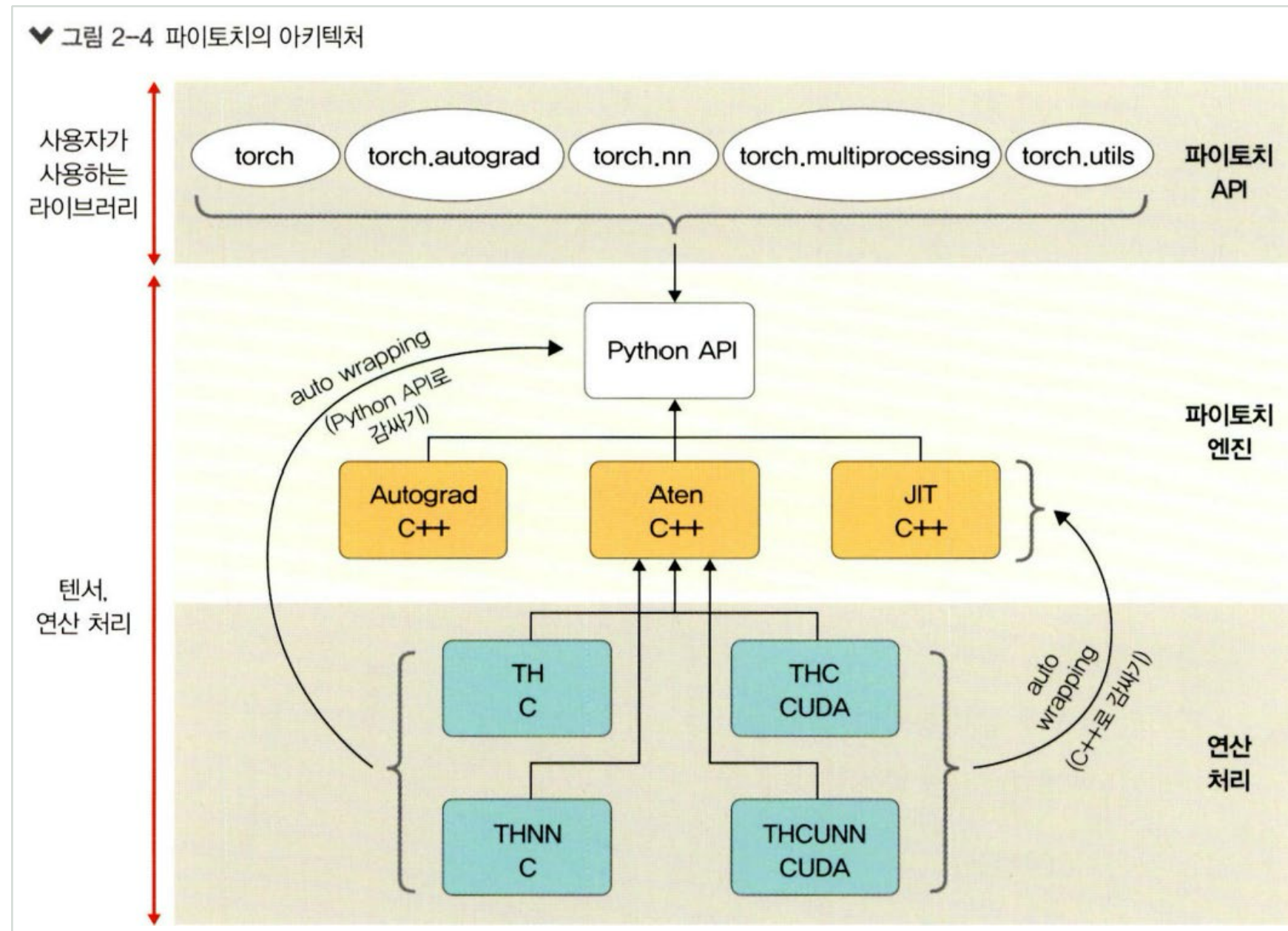


그림 2-4 파이토치의 아키텍처

① 파이토치 API (Python)

- 사용자가 직접 사용하는 라이브러리 계층
- 실제 계산은 하지 않고, C++ 엔진에 작업을 전달
 - `torch` : GPU를 지원하는 텐서 패키지, 다양한 수학 연산
 - `torch.autograd` : 자동 미분 패키지
 - `torch.nn` : 신경망 구축 · 훈련 (CNN, RNN, 정규화)
 - `torch multiprocessing` : 파이썬 멀티프로세싱
 - `torch.utils` : DataLoader 및 기타 유틸리티

2.1.2 아키텍처

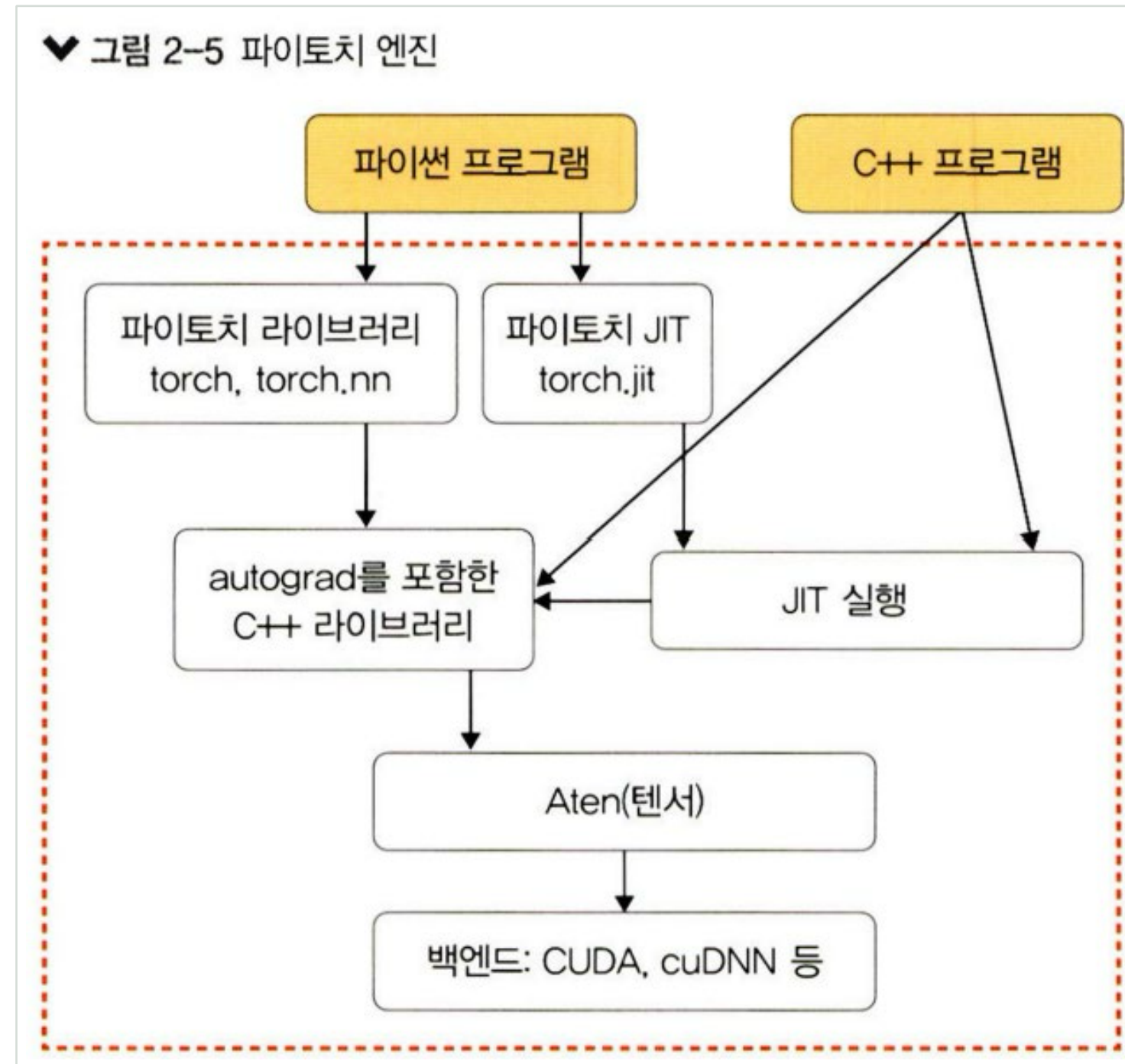


그림 2-5 파이토치 엔진

② 파이토치 엔진 (C++)

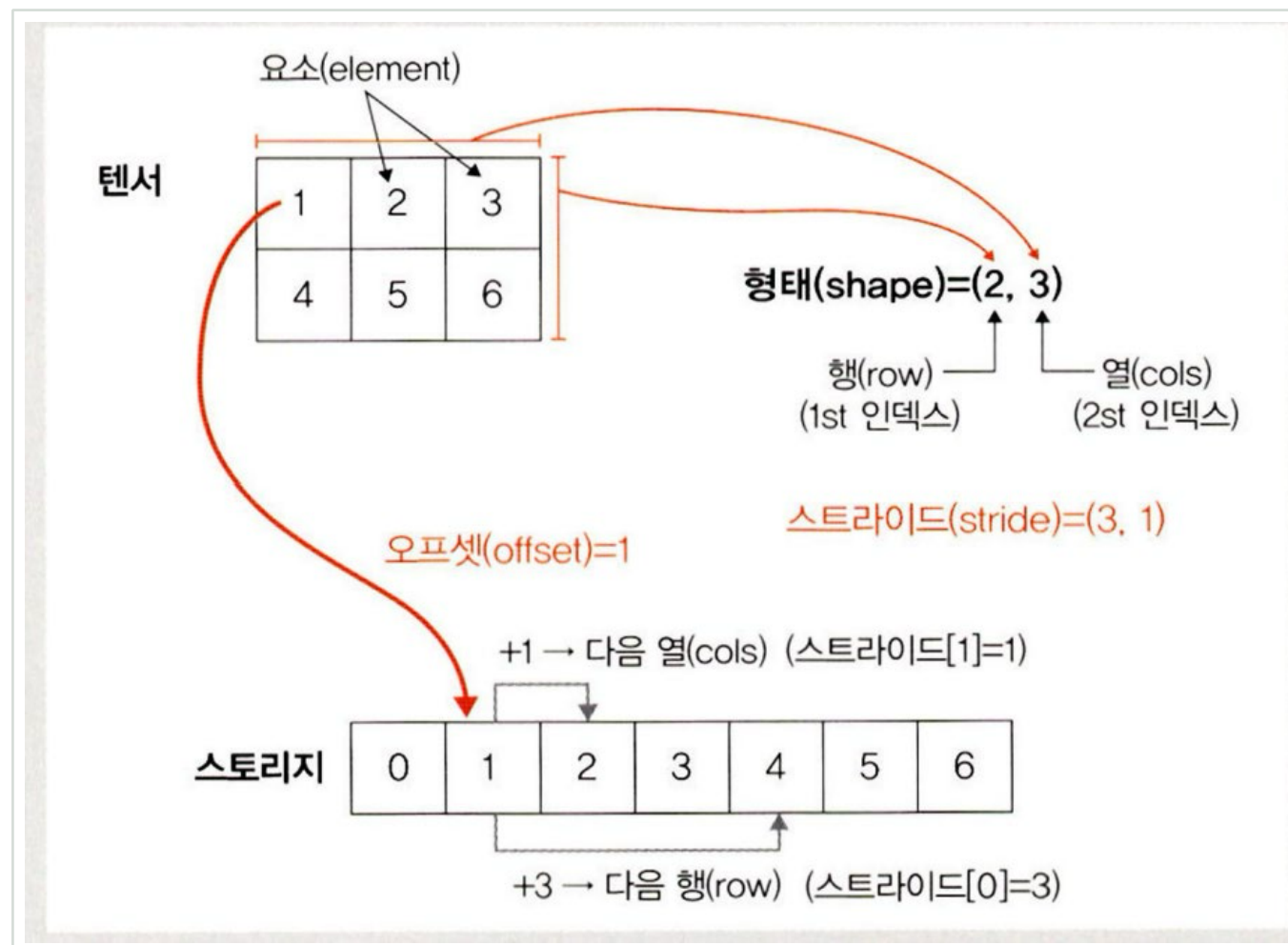
- 다차원 텐서와 자동 미분을 담당하는 핵심
 - Autograd C++ : 가중치 / 바이어스 업데이트에 필요한 미분 자동 계산
 - Aten C++ : C++ 텐서 라이브러리 (텐서 그 자체)
 - JIT C++ : 계산을 최적화하는 Just-In-Time 컴파일러
- Python API가 엔진을 감싸(auto wrapping) 파이썬에서 쓸 수 있게 해줌

③ 연산 처리 (C · CUDA)

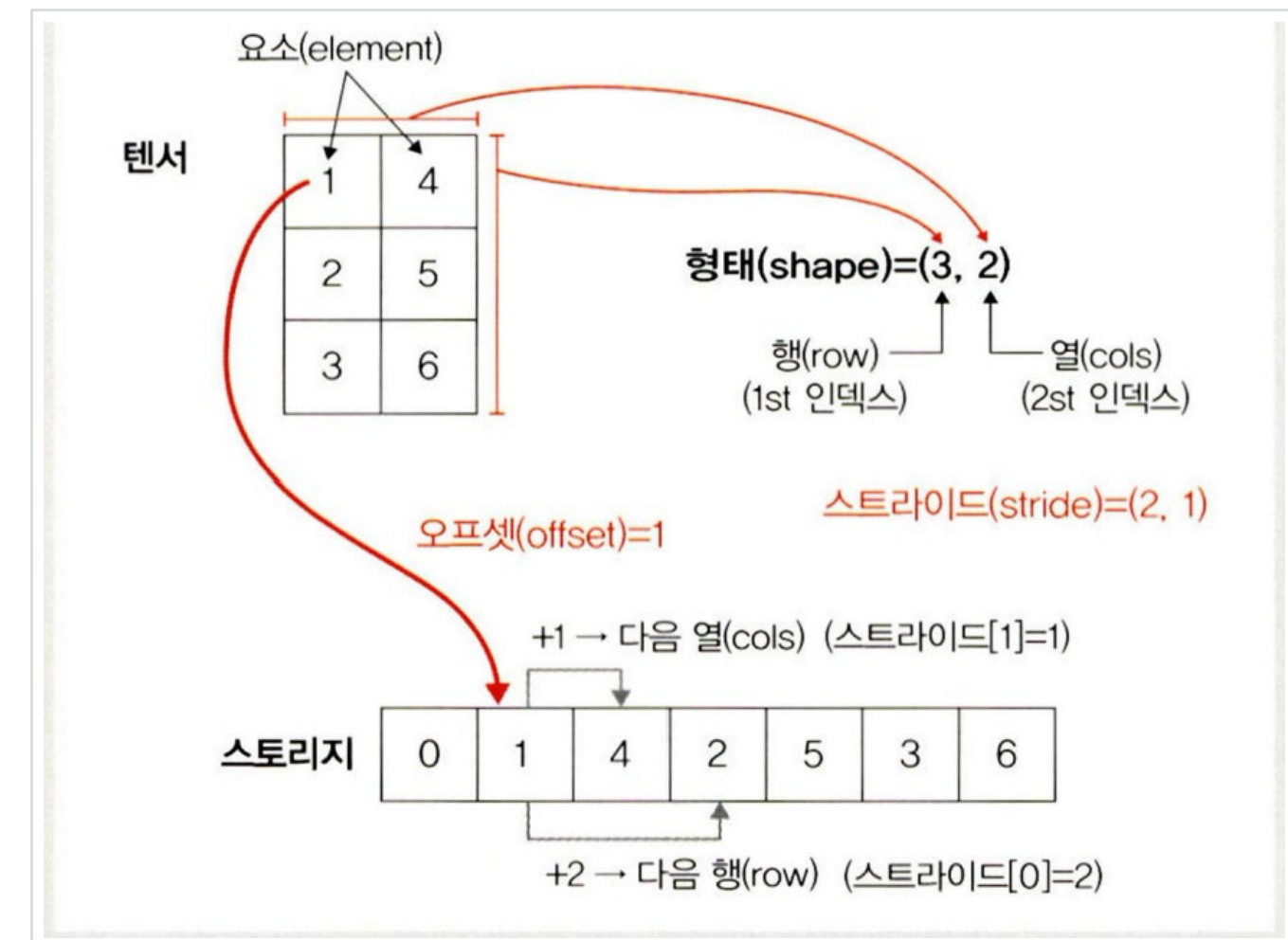
- 텐서의 실질적인 계산이 일어나는 최하위 계층
 - TH / THNN : CPU용 (C)
 - THC / THCUNN : GPU용 (CUDA)
- 효율적인 데이터 구조로 다차원 텐서 연산을 처리

2.1.2 스토리지 / 오프셋 / 스트라이드

- 텐서는 메모리에 저장될 때 1차원 배열이 되며, 이 배열을 스토리지(storage)라고 한다
- 오프셋(offset) : 텐서의 첫 번째 요소가 스토리지에 저장된 인덱스
- 스트라이드(stride) : 각 차원에서 다음 요소를 얻기 위해 건너뛰어야 하는 요소 개수



shape (2, 3) → stride (3, 1) : 다음 행은 3칸, 다음 열은 1칸



shape (3, 2) → stride (2, 1) : 형태가 바뀌면 스트라이드도 바뀐다

2.2 파이토치 기초 문법



2.2.1 텐서 다루기

```
import torch

# 2차원 형태의 텐서 생성
print(torch.tensor([[1, 2], [3, 4]]))

# GPU에 텐서 생성
print(torch.tensor([[1, 2], [3, 4]],
                    device="cuda:0"))

# dtype을 지정해 텐서 생성
print(torch.tensor([[1, 2], [3, 4]],
                    dtype=torch.float64))
```

torch.tensor()

- 파이썬 리스트/튜플/ndarray를 그대로 넣으면 텐서가 된다
- 중첩 리스트의 깊이가 곧 차원(dim)이 된다
 - `[[1, 2], [3, 4]]` → shape (2, 2)
- device : 텐서를 올릴 장치 (cpu / cuda:0)
- dtype : 요소의 자료형 지정
 - 지정하지 않으면 입력값에서 자동 추론 (정수 → int64)

자주 쓰는 생성 함수

- `torch.zeros(2, 3)` / `torch.ones(2, 3)`
- `torch.rand(2, 3)` : 0~1 균등분포 난수
- `torch.from_numpy(arr)` : 넘파이 → 텐서

2.2.1 텐서 다루기

텐서 → ndarray 변환

```
temp = torch.tensor([[1, 2], [3, 4]])
# 텐서를 ndarray로 변환
print(temp.numpy())

temp = torch.tensor([[1, 2], [3, 4]],
                    device="cuda:0")
# GPU 텐서 → CPU 텐서 → ndarray
print(temp.to("cpu").numpy())
```

- GPU에 있는 텐서는 .numpy()를 바로 쓸 수 없음
 - 넘파이는 CPU 메모리만 다루기 때문
- 반드시 .to("cpu")로 옮긴 뒤 변환한다

텐서의 자료형

torch.FloatTensor

32비트 부동 소수점 · 가장 많이 사용

torch.DoubleTensor

64비트 부동 소수점 · 높은 정밀도

torch.LongTensor

64비트 부호 있는 정수 · 분류 문제의 정답(label)

2.2.1 텐서 다루기

```
# 파이토치로 1차원 벡터 생성
temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7])

# 인덱스로 접근
print(temp[0], temp[1], temp[-1])

# 슬라이스로 접근
print(temp[2:5], temp[4:-1])
```

실행 결과

```
tensor(1.) tensor(2.) tensor(7.)
tensor([3., 4., 5.]) tensor([5., 6.])
```

인덱스 살펴보기

0	1	2	3	4	5	6
1.	2.	3.	4.	5.	6.	7.
-7	-6	-5	-4	-3	-2	-1

위 : 앞에서 센 인덱스 / 아래 : 뒤에서 센 인덱스

- 파이썬 리스트 · 넘파이와 인덱싱 방식이 같다
- -1은 마지막 요소를 의미한다
- temp[2:5] → 인덱스 2 이상 5 미만 → 3., 4., 5.
 - 끝 인덱스는 포함되지 않는다
- temp[4:-1] → 인덱스 4부터 마지막 직전까지

2.2.1 텐서 다루기

```
# 길이가 3인 벡터 생성
v = torch.tensor([1, 2, 3])
w = torch.tensor([3, 4, 6])

print(w - v)          # tensor([2, 2, 3])
print(v * w)          # 요소별 곱
print(torch.dot(v, w)) # 두 벡터의 내적
```

- ndarray처럼 다양한 수학 연산 사용 가능
- GPU에 올리면 같은 코드로 훨씬 빠르게 연산
- 브로드캐스팅도 넘파이와 동일하게 동작

타입 맞추어 연산

- FloatTensor(32비트)와 DoubleTensor(64비트)를 그대로 더하면 오류
- .float() / .double()로 타입을 맞춘 뒤 연산해야 함

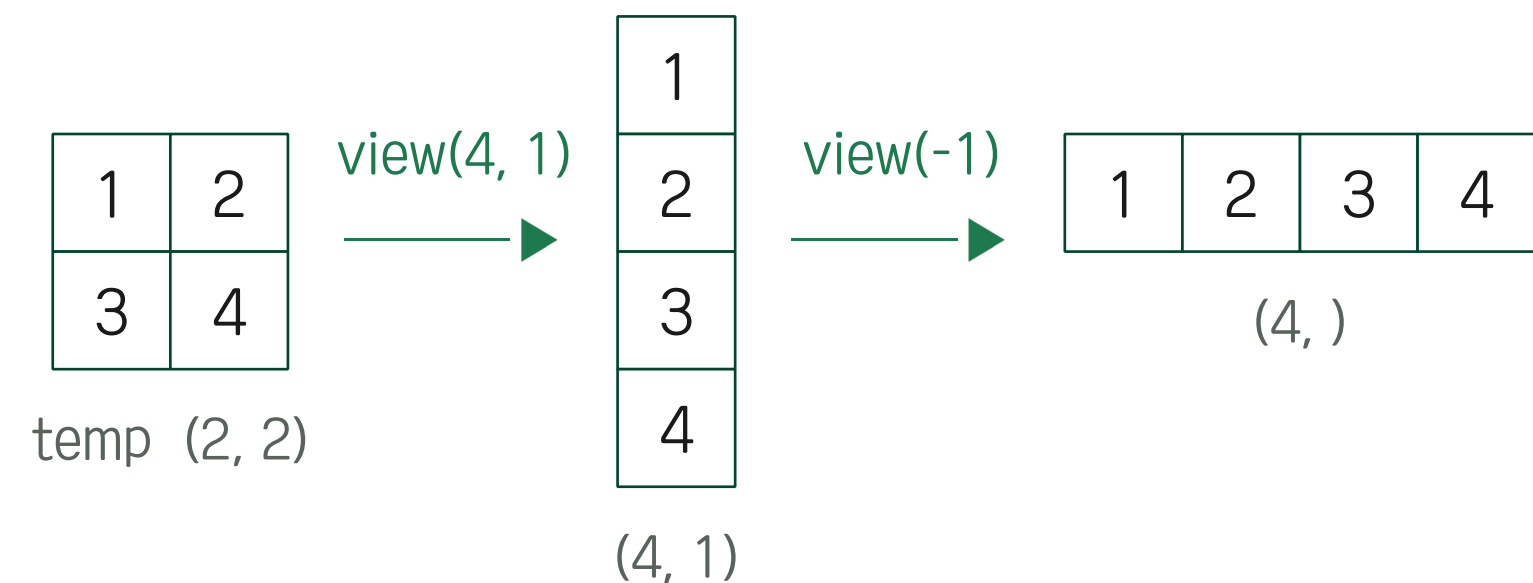
2.2.1 텐서 다루기

```
temp = torch.tensor([[1, 2], [3, 4]])
print(temp.shape)      # torch.Size([2, 2])

print(temp.view(4, 1))  # 4x1로 변형
print(temp.view(-1))    # 1차원 벡터로 변형
print(temp.view(1, -1)) # (1, 4)
print(temp.view(-1, 1)) # (4, 1)
```

- `view()`는 넘파이의 `reshape()`와 같은 역할
- 요소의 총 개수는 변하지 않는다 ($2 \times 2 = 4$ 개)
- `-1`은 '나머지 차원은 알아서 계산하라'는 의미
 - `view(1, -1) → (1, 4)` / `view(-1, 1) → (4, 1)`
- 스토리지는 그대로 두고 보는 방법(형태)만 바꿈

형태만 바꾼다



그 밖의 차원 조작

- **`torch.stack([a, b])`**
새로운 차원을 만들며 텐서를 쌓는다
- **`torch.cat([a, b], dim=0)`**
기존 차원을 따라 이어 붙인다
- **`t.t()` / `t.transpose(0, 1)`**
전치 · 지정한 두 차원의 순서를 바꾼다

2.2.2 데이터 준비

데이터를 준비하는 세 가지 방법

- ① 파이썬 라이브러리(pandas)로 파일을 그대로 불러오기
- ② 커스텀 데이터셋(Dataset)을 만들어 사용하기
- ③ 파이토치가 제공하는 데이터셋 사용하기

① 단순히 파일을 불러와 사용

```
# pip install pandas
import pandas as pd
import torch

data = pd.read_csv('../class2.csv')

# 칼럼 값을 넘파이 배열로 받아 텐서로 변환
x = torch.from_numpy(data['x'].values)\
    .unsqueeze(dim=1).float()
y = torch.from_numpy(data['y'].values)\
    .unsqueeze(dim=1).float()
```

- pandas로 CSV · JSON · 엑셀 파일을 그대로 불러올 수 있음
- .values → 넘파이 배열, from_numpy() → 텐서
- unsqueeze(dim=1) : (N,) → (N, 1) 로 차원을 늘려 입력 형태를 맞춤

2.2.2 데이터 준비

② 커스텀 데이터셋; 기본 틀

```
class CustomDataset(torch.utils.data.Dataset):  
  
    def __init__(self):  
        # 필요한 변수 선언, 데이터셋 전처리  
  
    def __len__(self):  
        # 데이터셋의 길이(총 샘플 수)  
  
    def __getitem__(self, index):  
        # index번째 데이터를 반환  
        # 반환값은 텐서 형태여야 한다
```

- 데이터를 한 번에 다 부르지 않고 조금씩 나누어 불러 사용
- 위 세 개의 함수를 반드시 구현
- 만든 Dataset을 DataLoader에 넘기면 배치 단위로 꺼내 쓸 수 있음

실제 사용 예

```
import pandas as pd, torch  
from torch.utils.data import Dataset, DataLoader  
  
class CustomDataset(Dataset):  
    def __init__(self, csv_file):  
        self.label = pd.read_csv(csv_file)  
  
    def __len__(self):  
        # 전체 크기 반환  
        return len(self.label)  
  
    def __getitem__(self, idx):  
        # idx번째 데이터  
        sample = torch.tensor(  
            self.label.iloc[idx, 0:3]).int()  
        label = torch.tensor(  
            self.label.iloc[idx, 3]).int()  
        return sample, label  
  
tensor_dataset = CustomDataset('../covtype.csv')  
dataset = DataLoader(tensor_dataset,  
                      batch_size=4, shuffle=True)
```

2.2.2 데이터 준비

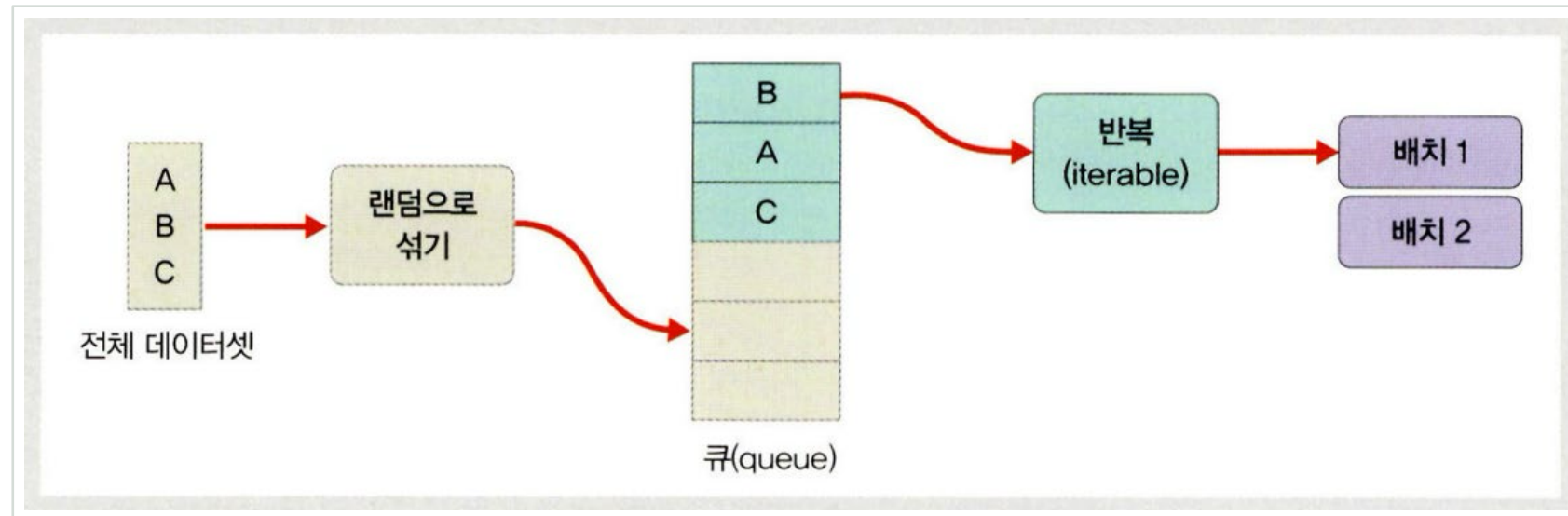


그림 2-6 torch.utils.data.DataLoader의 동작

torch.utils.data.DataLoader

- 학습에 사용될 데이터 전체를 보관했다가, 모델 학습을 할 때 배치 크기만큼 꺼내서 사용
- 데이터를 미리 잘라 두는 것이 아니라, 내부적으로 반복자(iterator)의 인덱스를 이용해 배치 크기만큼 반환
- shuffle=True 이면 매 에포크마다 순서를 랜덤으로 섞음

for 문으로 배치 꺼내 보기

```
for i, data in enumerate(dataset, 0):
    print(i, end='')
    batch = data[0]
    print(batch.size())
```

실행 결과

```
0torch.Size([4, 3])
1torch.Size([4, 3])
2torch.Size([4, 3])
3torch.Size([4, 3])
4torch.Size([3, 3])
```

- batch_size=4 → 한 번에 4개씩, 마지막 배치는 남은 3개만 반환됨

2.2.2 데이터 준비

③ 파이토치가 제공하는 데이터셋

- 토치비전(torchvision) : 파이토치가 제공하는 데이터셋이 모여 있는 패키지
- MNIST, CIFAR-10 등 유명한 데이터셋을 바로 내려받아 사용할 수 있음
- 전처리가 이미 되어 있어 실습용으로 편리
- transforms.Compose() 로 전처리 과정을 순서대로 묶음
 - ToTensor() : 이미지를 텐서로 변환 (0 ~ 1 범위로 스케일)
 - Normalize(평균, 표준편차) : 정규화
- train=True → 훈련 / train=False → 검증 · 테스트 데이터셋

MNIST 데이터셋 내려받기

```
import torchvision.transforms as transforms
from torchvision.datasets import MNIST

mnist_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (1.0,))
])

download_root = '../chap02/data/MNIST_DATASET'

train_dataset = MNIST(download_root,
    transform=mnist_transform, train=True, download=True)

valid_dataset = MNIST(download_root, # 검증(validation)
    transform=mnist_transform, train=False, download=True)

test_dataset = MNIST(download_root, # 테스트(test)
    transform=mnist_transform, train=False, download=True)
```

2.2.3 모델 정의

계층 · 모듈 · 모델

계층 (layer)

모듈을 구성하는 한 개의 계층 / 합성곱층, 선형 계층

모듈 (module)

한 개 이상의 계층이 모여 구성된 것

모델 (model)

최종적으로 원하는 네트워크

① 단순 신경망을 정의하는 방법

```
model = nn.Linear(in_features=1,
                  out_features=1, bias=True)
# nn.Module을 상속받지 않는 단순한 모델용
```

② nn.Module()을 상속하여 정의

```
class MLP(Module):
    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer = Linear(inputs, 1)  # 계층 정의
        self.activation = Sigmoid()     # 활성화 함수

    def forward(self, X):
        X = self.layer(X)
        X = self.activation(X)
        return X
```

- `__init__( )` : 모델에서 사용할 계층과 활성화 함수를 정의
- `forward( )` : 데이터가 흐르는 순서(연산)를 정의
- `model(x)` 를 호출하면 `forward( )`가 자동으로 실행된다

2.2.3 모델 정의

③ Sequential 신경망을 정의하는 방법

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=64,
                      kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=64, out_channels=30,
                      kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))
        self.layer3 = nn.Sequential(
            nn.Linear(in_features=30*5*5,
                      out_features=10, bias=True),
            nn.ReLU(inplace=True))

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = x.view(x.shape[0], -1)
        x = self.layer3(x)
        return x
```

- nn.Sequential 객체는 안에 포함된 모듈을 순차적으로 실행해 준다
- __init__()에서 네트워크를 묶어 두고 forward()에서는 실행 순서만 적으면 되므로 코드가 읽기 쉬워진다

2.2.3 모델 정의

④ 함수로 신경망을 정의하는 방법

```
def MLP(in_features=1, hidden_features=20,
        out_features=1):
    hidden = nn.Linear(in_features=in_features,
                       out_features=hidden_features, bias=True)
    activation = nn.ReLU()
    output = nn.Linear(
        in_features=hidden_features,
        out_features=out_features, bias=True)
    net = nn.Sequential(hidden, activation, output)
    return net
```

- 변수에 저장해 둔 계층들을 재사용할 수 있다는 장점
- 모델이 복잡해진다는 단점이 있어 복잡한 모델의 경우 nn.Module() 상속이 더 편리

2.2.4 모델 파라미터 정의

손실 함수 (loss function)

- 학습하는 동안 출력과 실제 값(정답) 사이의 오차를 측정
- $wX + b$ 로 계산한 값과 실제 값 y 의 오차를 구해 모델의 정확성을 측정

BCELoss

이진 분류(binary classification)를 위해 사용

CrossEntropyLoss

다중 클래스 분류를 위해 사용

MSELoss

회귀(regression) 모델에서 사용

옵티마이저 (optimizer)

- 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정
- `step()` 메서드로 전달받은 파라미터를 업데이트한다
- `zero_grad()` 는 파라미터들의 기울기를 0으로 만든다
- 파라미터별로 다른 기준(예 : 학습률)을 적용할 수 있다
 - `torch.optim.Optimizer(params, defaults)` 가 모든 옵티마이저의 기본 클래스
 - 종류 : Adam · SGD · RMSProp · Adagrad · Adadelta · Adamax · ASGD · LBFGS · Rprop

지표(metrics)

훈련과 테스트 단계를 모니터링하기 위해 사용

2.2.4 모델 파라미터 정의

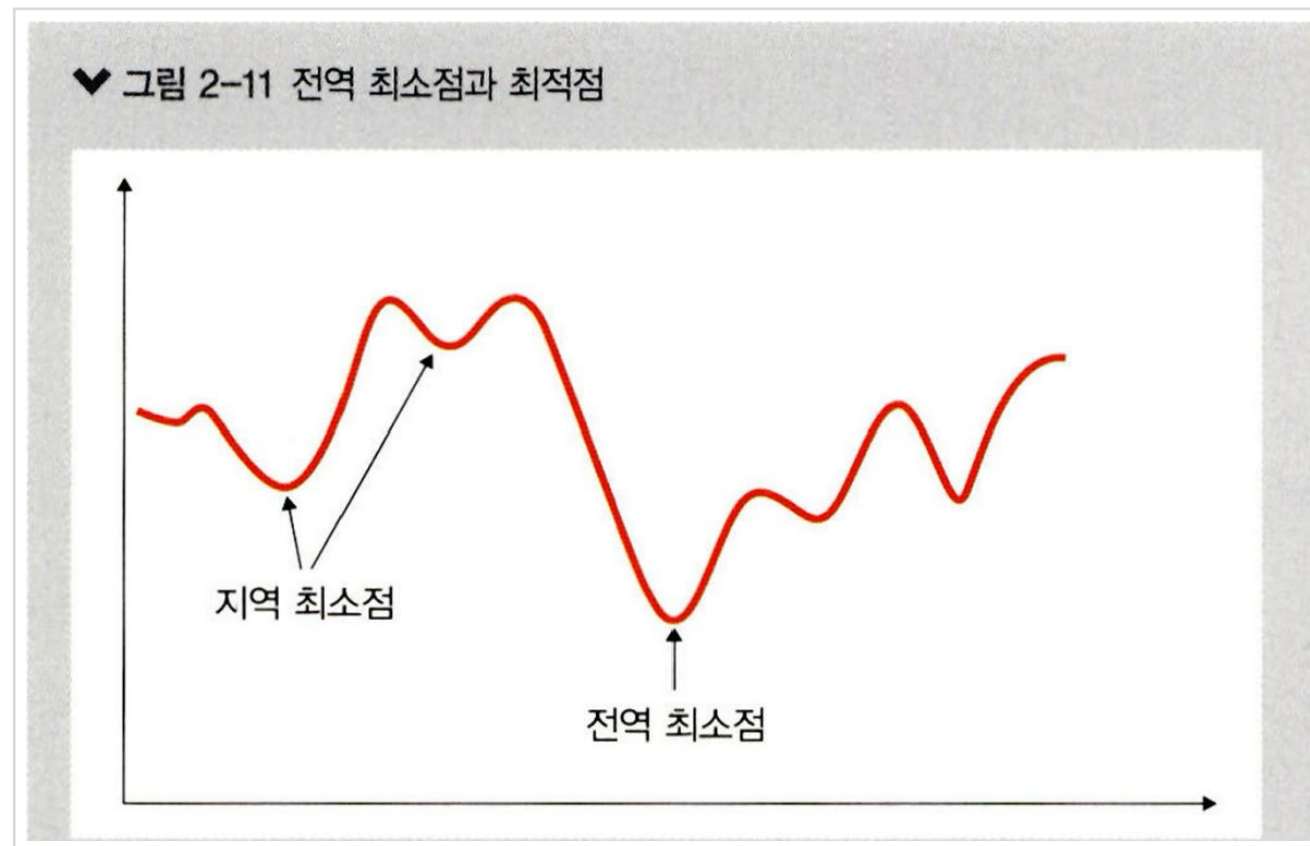


그림 2-11 전역 최소점과 지역 최소점

- 학습 목표 : 손실 함수의 값을 최소화하는 가중치와 바이어스를 찾는 것
- 전역 최소점(global minimum) : 오차가 가장 작을 때의 값 = 최적점
- 지역 최소점(local minimum) : 전역 최소점을 찾아가는 과정에서 만나는 hole

학습률 스케줄러 (lr scheduler)

- 지정한 횟수의 에포크를 지날 때마다 학습률을 감소 (decay)시켜 준다
- 초기에는 빠르게 학습하고, 전역 최소점 근처에서는 학습률을 줄여 최적점을 찾아감
 - LambdaLR : 람다 함수의 결과를 학습률로 설정
 - MultiStepLR : 지정된 에포크에만 감마 비율로 감소
 - CosineAnnealingLR : 코사인 함수 형태로 변화
 - ReduceLROnPlateau : 학습 상태에 따라 동적으로 변화

```
criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(),
                             lr=0.01, momentum=0.9)
scheduler = torch.optim.lr_scheduler.LambdaLR(
    optimizer=optimizer,
    lr_lambda=lambda epoch: 0.95 ** epoch)
```

2.2.5 모델 훈련

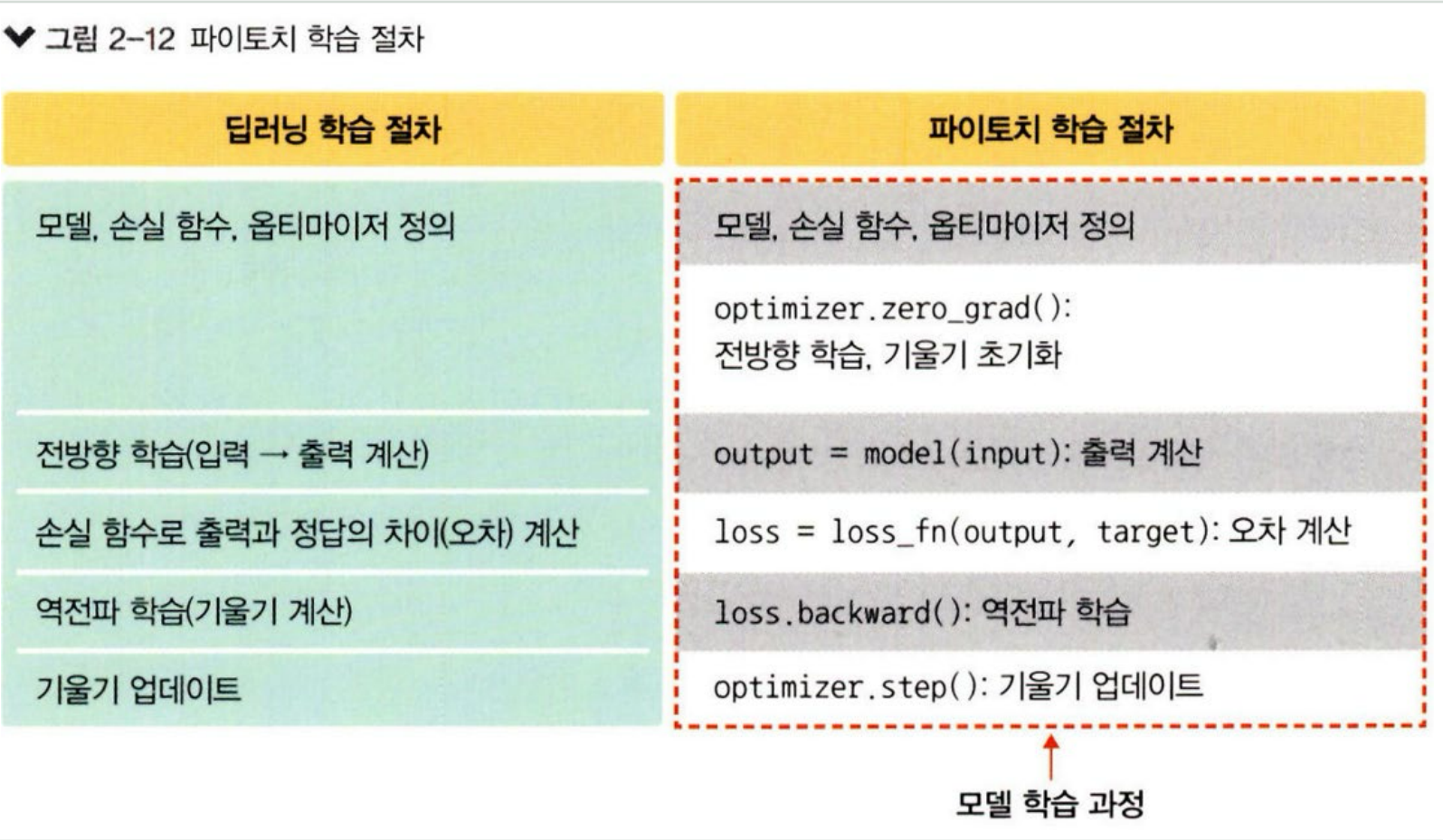


그림 2-12 딥러닝 학습 절차와 파이토치 코드

학습 = $y = wx + b$ 에서 적절한 w 와 b 값을 찾는 것

모델 훈련 코드

```
for epoch in range(100):
    yhat = model(x_train)           # 전방향 학습
    loss = criterion(yhat, y_train) # 오차 계산
    optimizer.zero_grad()          # 기울기 초기화
    loss.backward()                 # 역전파 학습
    optimizer.step()                # 기울기 업데이트
```

- 1. optimizer.zero_grad(): 이전 단계의 기울기 초기화
- 2. output = model(input): 순전파 연산으로 출력값 계산
- 3. loss_fn(output, target): 실제값과의 오차 계산
- 4. loss.backward(): 역전파를 통해 기울기(Gradient) 계산
- 5. optimizer.step(): 계산된 기울기를 바탕으로 파라미터 업데이트

2.2.6 모델 평가

① 함수를 이용해 평가

```
# pip install torchmetrics
import torch
import torchmetrics

preds = torch.randn(10, 5).softmax(dim=1)
target = torch.randint(5, (10,))

acc = torchmetrics.functional.accuracy(preds, target)
```

- torchmetrics 라이브러리로 정확도 등의 지표를 간단히 계산
- 함수 방식은 예측값과 정답을 한 번에 넣어 바로 계산
- 모델이 얼마나 잘 예측하는지 수치로 확인하는 단계

② 모듈을 이용해 평가

```
import torch
import torchmetrics

metric = torchmetrics.Accuracy()
n_batches = 10

for i in range(n_batches):
    preds = torch.randn(10, 5).softmax(dim=1)
    target = torch.randint(5, (10,))

    acc = metric(preds, target) # 배치별 정확도
    print(f"Accuracy on batch {i}: {acc}")

acc = metric.compute() # 전체 정확도
print(f"Accuracy on all data: {acc}")
```

- 모듈 방식은 배치마다 결과를 누적했다가 compute()로 전체 정확도를 계산
- 배치 단위로 학습하는 실제 상황에 적합

2.2.7 훈련 과정 모니터링

텐서보드(TensorBoard)

- 학습 과정(손실 · 정확도 등)을 그래프로 시각화해 모니터링하는 도구
- ① 텐서보드 설정 → ② 기록(write) → ③ 모델 구조 · 학습 곡선 확인

```
# pip install tensorboard
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("../chap02/tensorboard")

for epoch in range(num_epochs):
    model.train()
    for i, (x, y) in enumerate(dataloader):
        outputs = model(x)
        loss = criterion(outputs, y)
        writer.add_scalar("Loss", loss, epoch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

writer.close()

# 실행 : tensorboard --logdir=../chap02/tensorboard
#          --port=6006
```

model.train() & model.eval()

- model.train() : 훈련 데이터셋에 사용, 모델 훈련이 진행될 것임을 알림 → 드롭아웃 활성화
- model.eval() : 검증 · 테스트에 사용, 모든 노드를 사용하겠다는 의미 → 드롭아웃 비활성화
- torch.no_grad() : 기울기를 계산하지 않아 메모리와 속도에서 이득

```
model.eval()      # 검증 모드, dropout = False
with torch.no_grad():
    valid_loss = 0

    for x, y in valid_dataloader:
        outputs = model(x)
        loss = F.cross_entropy(
            outputs, y.long().squeeze())
        valid_loss += float(loss)

    valid_loss = valid_loss / len(valid_dataloader)
```

2.3 실습 환경 설정



#2.3 실습 환경 설정

#1 (2.3.1) 아나콘다 설치 : <https://www.anaconda.com/products/individual>



Products ▾Solutions ▾Resources ▾Company ▾

현조

Get Demo >

Get Started with Anaconda Distribution

Install Python, Jupyter, and thousands of data science packages in one step. Trusted by over 50 million users who need tools that work—without the setup headaches.

What's included in Anaconda Distribution? ^

✓ Thousands of vetted Python packages with dependency management

✓ Conda package and environment manager

✓ Jupyter Notebook and JupyterLab

✓ Desktop app (Navigator) or command line interface

✓ Works on Windows, macOS, and Linux

See [documentation](#) for more details and step-by-step instructions.

Download Now

Get access in 30 seconds.*

Download >

*Subject to our [Terms of Service](#). Use of Anaconda's offerings at an organization of more than 200 employees/contractors requires a paid business license unless your organization is eligible for discounted or free use. [See Pricing](#).

Hi, how can I help?

1



EWHA
EURO

#2.3 실습 환경 설정

#2 (2.3.2) 가상 환경 생성

```
Anaconda Prompt
(base) C:\Users\user>conda create -n torch_book python=3.9.0
3 channel Terms of Service accepted
Retrieving notices: done
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

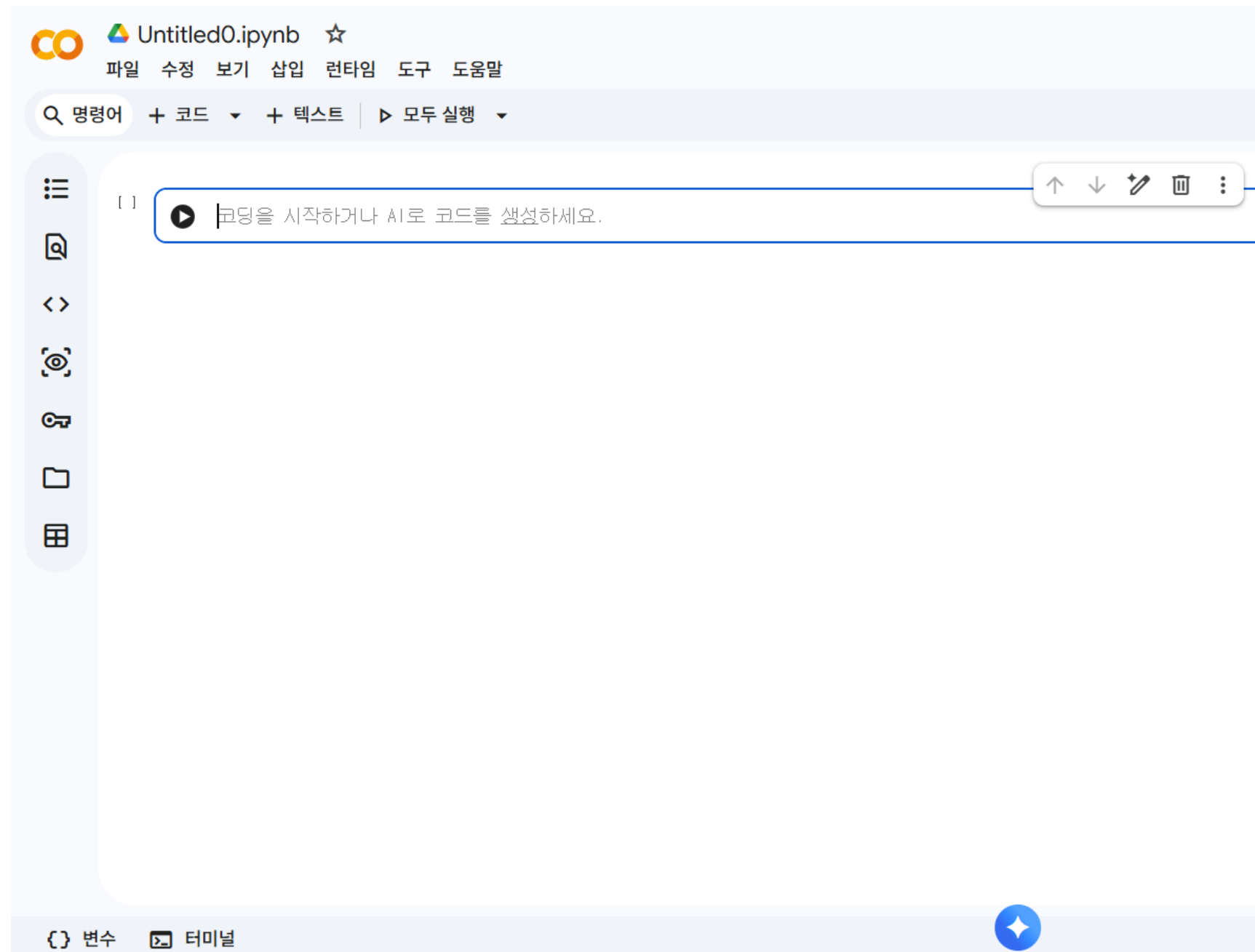
==> WARNING: A newer version of conda exists. <==
current version: 26.1.1
latest version: 26.7.2
```

1. Anaconda prompt 실행
2. 가상 환경 생성 (이름 : torch_book)
: conda create -n torch_book python=3.9.0
3. 가상 환경 확인
: conda env list
4. 가상 환경 활성화
: activate torch_book

- 가상 환경을 잘못 만들어서 삭제하고 싶은 경우
: conda env remove -n torch_book
- 가상 환경에 커널 설치 및 연결
: conda install ipykernel
: ipython kernel install --name tf2_book --user

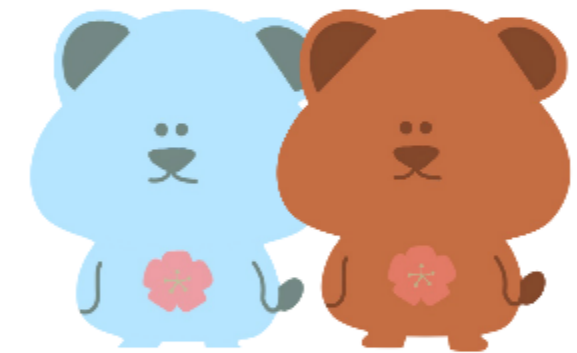
#2.3 실습 환경 설정

#3 코랩 활용



- 코랩 : 클라우드 기반 무료 주피터 노트북 개발 환경
 - 딥러닝 모델을 활용하여 데이터를 분석할 경우
다량의 데이터를 다루게 됨
- > 고성능 PC 환경을 개인이 갖추기는 어렵기 때문에
코랩을 많이 사용함

2.4 파이토치 코드 맛보기



#2.4 파이토치 코드 맛보기 : 차 상태 예측

#1 데이터셋 (CSV파일) 내려받기, 라이브러리 호출 – 코랩 활용

1	price	maint	doors	persons	lug_capacity	safety	output
2	vhigh	vhigh	2	2	small	low	unacc
3	vhigh	vhigh	2	2	small	med	unacc
4	vhigh	vhigh	2	2	small	high	unacc
5	vhigh	vhigh	2	2	med	low	unacc

- Price : 자동차 가격
- Maint : 자동차 유지 비용
- Doors : 자동차 문 개수
- Persons : 수용 인원
- Lug_capacity : 수하물 용량
- Safety : 안전성
- Output : 차 상태
 - unacc(허용 불가능한 수준)
 - Acc (허용 가능한 수준)
 - Good (양호)
 - Very good, vgood (매우 좋은)

```
import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
from google.colab import files # 데이터 불러오기
file_uploaded=files.upload() # 데이터 불러오기
dataset = pd.read_csv('car_evaluation.csv')
```

파일 선택 car_evaluation.csv

car_evaluation.csv(text/csv) - 51920 bytes, last modified: 2026. 9. 7. - 100% done

Saving car_evaluation.csv to car_evaluation.csv

- 라이브러리 호출
- 데이터 호출

#2.4 파이토치 코드 맛보기 : 차 상태 예측

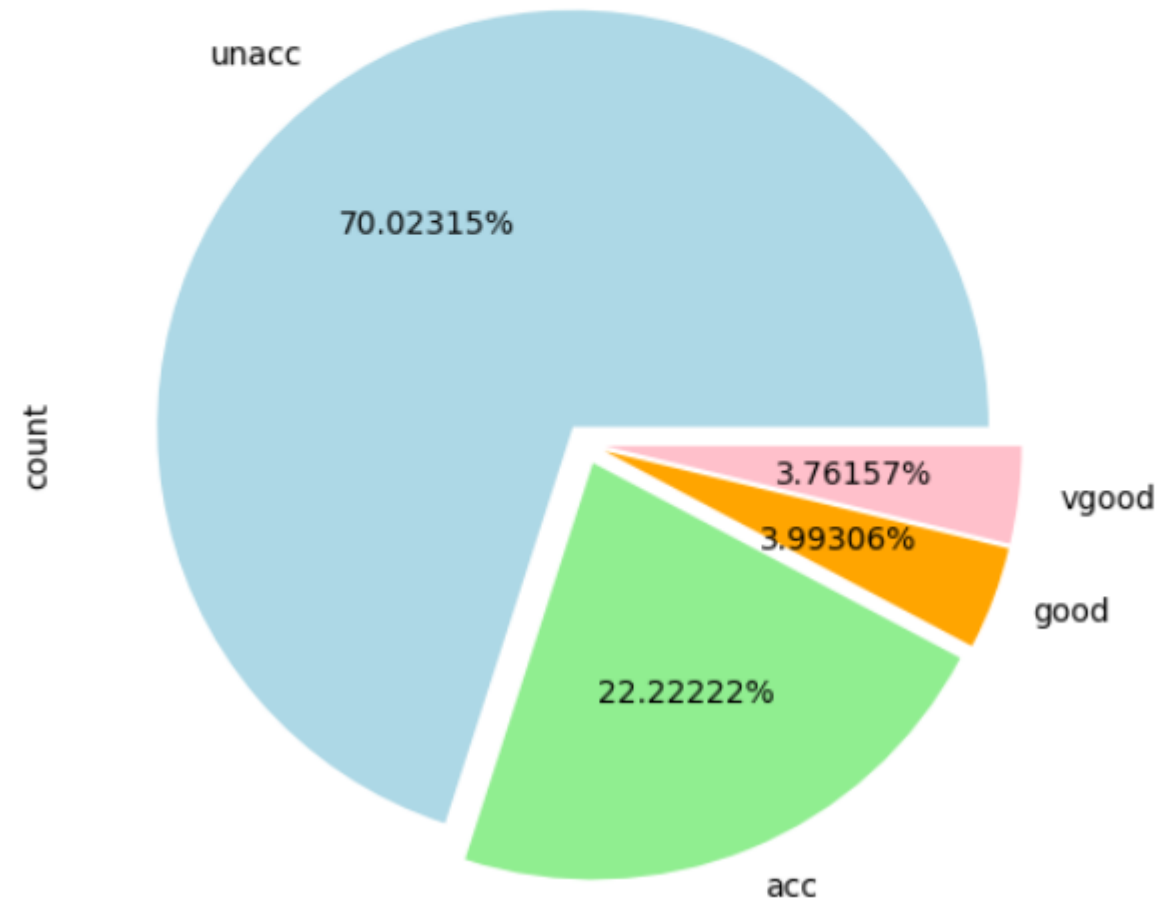
#2 데이터셋 분포 시각화

```
dataset.head()
```

	price	maint	doors	persons	lug_capacity	safety	output
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

```
fig_size = plt.rcParams["figure.figsize"]
fig_size[0] = 8
fig_size[1] = 6
plt.rcParams["figure.figsize"] = fig_size
dataset.output.value_counts().plot(kind='pie', autopct='%0.05f%%',
                                   colors=['lightblue', 'lightgreen', 'orange', 'pink'],
                                   explode=(0.05, 0.05, 0.05,0.05))

... <Axes: ylabel='count'>
```



- 대부분의 자동차(70%)는 output이 unacc 상태
-> 양호한 상태의 자동차 비율이 매우 적다.

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#3 데이터 전처리(1) – 데이터를 범주형 타입으로 변환

절차 : 범주형 데이터 -> dataset[category] -> 넘파이 배열 -> 텐서

```
categorical_columns = ['price', 'maint', 'doors', 'persons', 'lug_capacity', 'safety']
```

```
for category in categorical_columns:  
    dataset[category] = dataset[category].astype('category')
```

```
price = dataset['price'].cat.codes.values  
maint = dataset['maint'].cat.codes.values  
doors = dataset['doors'].cat.codes.values  
persons = dataset['persons'].cat.codes.values  
lug_capacity = dataset['lug_capacity'].cat.codes.values  
safety = dataset['safety'].cat.codes.values
```

```
categorical_data = np.stack([price, maint, doors, persons, lug_capacity, safety], 1)  
categorical_data[:10]
```

```
array([[3, 3, 0, 0, 2, 1],  
       [3, 3, 0, 0, 2, 2],  
       [3, 3, 0, 0, 2, 0],  
       [3, 3, 0, 0, 1, 1],  
       [3, 3, 0, 0, 1, 2],  
       [3, 3, 0, 0, 1, 0],  
       [3, 3, 0, 0, 0, 1],  
       [3, 3, 0, 0, 0, 2],  
       [3, 3, 0, 0, 0, 0],  
       [3, 3, 0, 1, 2, 1]], dtype=int8)
```

- 데이터셋 칼럼들의 목록
- astype() 메서드 활용 -> 범주형으로 변환
- 범주형 데이터를 숫자로 변환 (cat.codes)
- 변환된 숫자(넘파이 배열)을 합침 -> 2차원 배열
- 합친 넘파이 배열 중 10개의 행을 출력

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#3 데이터 전처리(2) – 배열을 텐서로 변환

```
categorical_data = torch.tensor(categorical_data, dtype=torch.int64)
categorical_data[:10]
```

```
tensor([[3, 3, 0, 0, 2, 1],
        [3, 3, 0, 0, 2, 2],
        [3, 3, 0, 0, 2, 0],
        [3, 3, 0, 0, 1, 1],
        [3, 3, 0, 0, 1, 2],
        [3, 3, 0, 0, 1, 0],
        [3, 3, 0, 0, 0, 1],
        [3, 3, 0, 0, 0, 2],
        [3, 3, 0, 0, 0, 0],
        [3, 3, 0, 1, 2, 1]])
```

```
outputs = pd.get_dummies(dataset.output)
outputs = outputs.values
outputs = torch.tensor(outputs).flatten()
```

```
print(categorical_data.shape)
print(outputs.shape)
```

```
torch.Size([1728, 6])
torch.Size([6912])
```

- torch 모듈을 이용하여 배열을 텐서로 변환

- Label로 활용한 칼럼에 대해서도 텐서로 변환
-> get_dummies 이용 : 문자를 (0, 1)로 바꾸어 줌

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#4 train set, test set 분리

```
categorical_column_sizes = [len(dataset[column].cat.categories) for column in categorical_columns]
categorical_embedding_sizes = [(col_size, min(50, (col_size+1)//2)) for col_size in categorical_column_sizes]
print(categorical_embedding_sizes)
```

```
[(4, 2), (4, 2), (4, 2), (3, 2), (3, 2), (3, 2)]
```

```
total_records = 1728
test_records = int(total_records * .2)
```

```
categorical_train_data = categorical_data[:total_records-test_records]
categorical_test_data = categorical_data[total_records-test_records:total_records]
train_outputs = outputs[:total_records-test_records]
test_outputs = outputs[total_records-test_records:total_records]
```

```
print(len(categorical_train_data))
print(len(train_outputs))
print(len(categorical_test_data))
print(len(test_outputs))
```

```
• 1383
  1383
  345
  345
```

- 범주형 칼럼을 N차원으로 변환

- 데이터셋 분리 : 20%를 테스트용으로 사용

- 데이터셋 분리 확인
-> 훈련 및 테스트용 개수 출력

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#5 모델의 네트워크 생성 & 객체 생성

```
class Model(nn.Module):
    def __init__(self, embedding_size, output_size, layers, p=0.4):
        super().__init__()
        self.all_embeddings = nn.ModuleList([nn.Embedding(ni, nf) for ni, nf in embedding_size])
        self.embedding_dropout = nn.Dropout(p)

        all_layers = []
        num_categorical_cols = sum((nf for ni, nf in embedding_size))
        input_size = num_categorical_cols

        for i in layers:
            all_layers.append(nn.Linear(input_size, i))
            all_layers.append(nn.ReLU(inplace=True))
            all_layers.append(nn.BatchNorm1d(i))
            all_layers.append(nn.Dropout(p))
            input_size = i

        all_layers.append(nn.Linear(layers[-1], output_size))
        self.layers = nn.Sequential(*all_layers)

    def forward(self, x_categorical):
        embeddings = []
        for i, e in enumerate(self.all_embeddings):
            embeddings.append(e(x_categorical[:, i]))
        x = torch.cat(embeddings, 1)
        x = self.embedding_dropout(x)
        x = self.layers(x)
        return x
```

```
model = Model(categorical_embedding_sizes, 4, [200, 100, 50], p=0.4)
print(model)
```

- 모델에서 사용되는 파라미터와 신경망 초기화
- For문 활용 : 각 계층을 all_layers 목록에 추가
 - Linear : 데이터에 선형 변환 진행
 - ReLU : 활성화 함수로 사용
 - BatchNorm1d : 배치 정규화 용도로 사용
 - Dropout : 과적합 방지에 사용
- forward() : 연산 진행

- 객체 생성 : 임베딩 크기, 출력 크기, 은닉층의 뉴런, 드롭아웃 전달

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#6 손실 함수, 옵티마이저 정의 & CPU/GPU 사용 지정

```
loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
if torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')
```

- 크로스 엔트로피 사용 : 데이터 분류
- GPU가 있다면 GPU를 사용하도록 함
(파이토치 – GPU에 최적화된 딥러닝 프레임워크)

#7 모델 학습

```
epochs = 500
aggregated_losses = []
train_outputs = train_outputs.to(device=device, dtype=torch.int64)
for i in range(epochs):
    i += 1
    y_pred = model(categorical_train_data).to(device)
    single_loss = loss_function(y_pred, train_outputs)
    aggregated_losses.append(single_loss)

    if i%25 == 1:
        print(f'epoch: {i:3} loss: {single_loss.item():10.8f} ')

    optimizer.zero_grad()
    single_loss.backward()
    optimizer.step()

print(f'epoch: {i:3} loss: {single_loss.item():10.10f}')
```

- For문은 500회 반복됨
: 각 반복마다 손실함수가 오차 계산
- 옵티마이저 함수의 step() 메서드를 통해 기울기 업데이트

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#8 테스트 데이터셋으로 모델 예측

```
test_outputs = test_outputs.to(device=device, dtype=torch.int64)
with torch.no_grad():
    y_val = model(categorical_test_data).to(device)
    loss = loss_function(y_val, test_outputs)
print(f'Loss: {loss:.8f}')
```

```
Loss: 0.55816710
```

```
print(y_val[:5])
```

```
tensor([[ 1.5791,  0.8695, -2.0905, -1.9746],
        [ 2.3801,  1.2895, -3.7661, -3.5672],
        [ 2.7393,  1.5828, -3.9005, -3.7467],
        [ 2.9840,  2.0176, -4.0020, -4.0287],
        [ 2.7245,  1.8273, -3.2561, -3.0924]], device='cuda:0')
```

```
y_val = np.argmax(y_val.cpu().numpy(), axis=1)
print(y_val[:5])
```

```
[0 0 0 0 0]
```

- 테스트 데이터셋으로 모델 예측

- 모델의 예측 확인

- 가장 큰 값을 갖는 인덱스 확인

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#8 평가 지표 활용

- 정확도 : 전체 예측 건수에서 정답을 맞힌 건수의 비율

$$\frac{\text{True Positive} + \text{True Negative}}{\text{True Positive} + \text{True Negative} + \text{False Positive} + \text{False Negative}}$$

- 재현율 : 실제로 정답이 1일때 모델도 1로 예측한 비율

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

- 정밀도 : 모델이 1로 예측한 것 중에서 실제로 정답이 1인 비율

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

- F1-스코어 : 정밀도와 재현율의 조화 평균

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

#2.4 파이토치 코드 맛보기 : 차 상태 예측

#8 평가 지표 활용

```
import warnings
warnings.filterwarnings('ignore')
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

test_outputs=test_outputs.cpu().numpy()
print(confusion_matrix(test_outputs,y_val))
print(classification_report(test_outputs,y_val))
print(accuracy_score(test_outputs, y_val))
```

```
[[258  1]
 [ 86  0]]
```

	precision	recall	f1-score	support
0	0.75	1.00	0.86	259
1	0.00	0.00	0.00	86
accuracy			0.75	345
macro avg	0.38	0.50	0.43	345
weighted avg	0.56	0.75	0.64	345

```
0.7478260869565218
```

- 약 75%의 정확도를 보임
- 추후 파라미터를 변경하며 더 나은 성능을 찾아볼 수 있음

THANK YOU

